rxlibs

numpy, scipy and pandas style native plugins for CREXX Level B

Reference Manual and User’s Guide

Release 1 • Build 4 • 20 October 2026
for CREXX Level B with the plugin interface rxpa, and any C compiler

Publication RXL-0001-0
File No. RXLIBS-01

First Edition (October 2026)

This edition applies to rxlibs Release 1 (build 4, 20 October 2026), a set of native plugins (rxnumpy, rxscipy, rxpandas) and a Level B module (rxspalg) for CREXX, and to all subsequent builds until otherwise indicated in new editions.

rxlibs is a hobbyist project. It is not a product of, and this manual is not a publication of, the NumPy, SciPy or pandas projects or of the Python Software Foundation. NumPy, SciPy, pandas and Python are names of libraries and languages, used here only to identify the libraries whose interfaces and results rxlibs follows and the implementations to which it is compared.

CREXX is an open-source REXX implementation maintained by its own community. The plugins are built against its public plugin interface (rxpa) and run on an unmodified CREXX virtual machine of the same version.

The project repository holds the source of the plugins, the regression tests and the benchmark programs from which every statement in this manual can be checked. Comments on this publication may be addressed to the project.

Contents

Preface 1

Chapter 1. Concept and Architecture 2
- The components 2
- Plugin or Python? 2
- Architecture 2
- Why the algorithms are the way they are 3

Chapter 2. Installing, Building and Running 4
- Requirements 4
- Build 4
- Compile and run a program 4
- Running the tests 5

Chapter 3. The Handle Model and Error Signalling 6
- Handles 6
- Shapes, text and complex numbers 6
- Out parameters 6
- Errors 7
- Memory and threads 7

Chapter 4. Reference: rxnumpy 8
- Creating arrays 8
- Properties and element access 9
- Shape manipulation 10
- Element-wise arithmetic 10
- Element-wise mathematical functions 12
- Reductions 13
- Cumulative functions and sorting 14
- Linear algebra 15
- Fourier transform 16
- Legacy random numbers (MT19937) 16
- Generator (default_rng, PCG64) 17
- Comparison, masks and indexing 18

Chapter 5. Reference: rxscipy 20
- Distributions 20
- Random variates 23
- Scalar mathematics 20
- Special functions 21
- Normal and Student t shortcuts 22
- Distributions: pdf, pmf, cdf, sf, ppf, mean, var 22
- Random variates 23
- Integration, interpolation and polynomials 23
- Descriptive statistics 24
- Statistical tests 24

Chapter 6. Reference: rxpandas 26
- Creating and inspecting tables 26
- Selecting and ordering 27
- Grouping, merging and reshaping 27
- Index 28
- Date and time 29

Chapter 7. Callbacks and the Solver Reference 30
- The Level B callback convention 30
- Solver reference 30

Chapter 8. Worked Examples 32
- Linear regression 32
- A differential equation 32
- Optimisation 33
- Dataframe groupby 35
- Statistical tests and distributions 37

Chapter 9. Limits 39
- What is not implemented 39
- Numerical agreement 39

Chapter 10. Quirks of Level B 40

Chapter 11. Performance **41**
Reading the table 41
Appendix A. Test Programs **43**
Appendix B. Files of the Repository **44**
Appendix C. Accuracy Against scipy **45**

Preface

This manual describes rxlibs, a set of three native plugins for the CREXX Level B language that provide numpy-, scipy- and pandas-style numerical computing without a Python interpreter: rxnumpy (n-dimensional arrays, linear algebra, FFT, random numbers), rxscipy (special functions, probability distributions, statistics, interpolation) and rxpandas (column tables with CSV, groupby, merge and pivot). A fourth component, the Level B module rxspalg, adds the algorithms that need a callback into the user's program: root finding, integration, minimisation, differential equations and curve fitting.

It is written for the CREXX programmer who needs numerical work to run at native speed, and for the Python programmer who wants to know which parts of numpy, scipy and pandas exist in this form and how closely they agree with the originals. Every number printed in this manual was produced by running the example on the build named on the title page; the checker examples_check.py re-runs all of them.

How this manual is organized

- **Chapter 1, Concept and Architecture**, explains what the plugins are, why they are plugins and not Python, and how they relate to the Python-on-CREXX interpreter.
- **Chapter 2, Installing, Building and Running**, covers the build scripts, the environment variables and how a program is compiled and run.
- **Chapter 3, The Handle Model and Error Signalling**, describes how arrays and tables are passed around, what is shared and what is copied, out parameters and the signals raised on errors.
- **Chapters 4, 5 and 6** are the function reference of rxnumpy (np_*), rxscipy (sp_*) and rxpandas (pd_*): name, arguments, return type, description, and an example with the result obtained by running it.
- **Chapter 7, Callbacks and the Solver Reference**, describes the Level B callback convention and the solvers of rxspalg.
- **Chapter 8, Worked Examples**, solves four complete problems: linear regression, a differential equation, optimisation and a dataframe groupby, plus statistical tests.
- **Chapter 9, Limits**, lists what is not implemented and the numerical agreement with numpy and scipy. **Chapter 10, Quirks of Level B**, lists the language pitfalls met while writing the libraries.
- **Chapter 11, Performance**, gives the benchmark of the plugins against CPython numpy, scipy and pandas.
- **Appendix A** lists the test programs, **Appendix B** the files of the repository, **Appendix C** the accuracy test against scipy.

Conventions

- Program text and output are shown in a fixed font. In a listing, a line beginning with $ rxrun names the program that produced the output below it. rxrun is scripts/run.sh of the repository.
- A type written .binary is a handle to an array or a table, .float a double, .int a 64-bit integer, .string a character string.
- Name prefixes: np_ belongs to rxnumpy, sp_ to rxscipy and rxspalg, pd_ to rxpandas. A call is written namespace..function, for example rxnumpy..np_sum(a).

Chapter 1. Concept and Architecture

rxlibs is a C89 numerical core wrapped in three CREXX plugins. A CREXX Level B program imports a plugin namespace and calls its functions like any other Level B function. The arrays and tables live in C memory; the program only holds opaque handles to them. No Python interpreter, no byte code and no dynamic typing is involved: a call such as rxnumpy..np_dot(a, b) is a direct call of a C function through the CREXX plugin interface (rxpa).

The components

Component	Namespace	Content
rxnumpy	rxnumpy..np_*	dense float64 and int64 n-dimensional arrays, broadcasting, reductions, sort, matmul, slicing, boolean and integer indexing, linear algebra, FFT, legacy MT19937 and default_rng (SeedSequence, PCG64, ziggurat) random numbers
rxscipy	rxscipy..sp_*	special functions, 12 probability distributions, descriptive statistics and tests, trapz and simpson, interpolation, cubic splines, polynomial fits
rxspalg	rxspalg..sp_*	Level B module (source rxscipy/lib/rxspalg.crexx) with the callback algorithms: brentq, newton, quad, minimize (Nelder-Mead, BFGS), solve_ivp (RK45), curve_fit (Levenberg-Marquardt)
rxpandas	rxpandas..pd_*	column tables: CSV, filter, sort, groupby, merge, value_counts, pivot, index, ISO date and time, describe

Figure 1. The components of rxlibs

Plugin or Python?

The repository also contains Python/CREXX, an interpreter for Python 3 written in CREXX, which has its own numpy, pandas and scipy subsets written in Python. rxlibs is the alternative for programs that are CREXX programs to begin with: there is no second language, no interpreter start-up and no per-element interpretation. The two are compared in the next table; the measured speed difference is in Chapter 11 and in the README of the repository.

	rxlibs plugins	Python/CREXX numpy, pandas, scipy
Caller	CREXX Level B program	Python program run by the interpreter written in CREXX
Where the work is done	C functions in a plugin library	Python code interpreted by CREXX
Data	opaque handles to C arrays	Python objects
Numerics	float64 and int64 arrays, bit-compatible random streams	pure Python emulation of the libraries
Callbacks (minimize, quad, ...)	objects implementing library interfaces, solvers written in Level B	ordinary Python functions
Speed	native for the array kernels	interpreted
Best for	numerical work inside CREXX applications	running existing Python code

Figure 2. rxlibs plugins compared with Python on CREXX

Architecture

The C code is split in libraries that know nothing about CREXX or Python (the core directory), and thin plugin wrappers (one C file per plugin) that convert arguments and raise signals.

```
CREXX Level B program
      |  rxnumpy..np_dot(a, b)           import rxnumpy / rxscipy / rxpandas / rxspalg
      v
plugin wrapper   rxnumpy/plugin/rxnumpy.c      argument checks, handle <-> object,
    RETURNSIGNAL
      v
C89 core         core/rxnp.c rxnp_la.c rxnp_lapack.c rxnp_fft.c rxnp_rng.c rxnp_pcg.c
    rxnp_idx.c
                 core/rxsp.c rxsp_dist.c       special functions, distributions,
                     statistics
                 core/rxdf.c                   tables
```

```
Level B module   rxscipy/lib/rxspalg.crexx     solvers calling back into user objects
```

Figure 3. Layers of rxlibs

Each plugin is one shared library with the extension .rxplugin, loaded by the CREXX virtual machine. All three plugins compile the same array code (rxnp.c), so an rxscipy or rxpandas function accepts an rxnumpy array handle: the payload carries a type name that the other plugin recognises, and the array structure is identical.

Why the algorithms are the way they are

- Random numbers are bit-for-bit those of numpy: the legacy np.random.seed stream (MT19937) and default_rng (SeedSequence, PCG64 and the ziggurat tables). A seeded program and a seeded Python program print the same numbers.
- Sums use numpy's pairwise summation, so sums agree with numpy bit for bit; the matrix product is a plain triple loop.
- The solvers follow scipy's algorithms (brentq.c, QUADPACK qk21, the Nelder-Mead simplex, RK45 with scipy's step control, Levenberg-Marquardt), so that results agree with scipy to the printed digits. They are written in Level B because a native plugin cannot call a Level B procedure.

Chapter 2. Installing, Building and Running

Requirements

- A C compiler (gcc or cc) for the plugins. The core is strict C89 apart from the unsigned long long behaviour of the 64-bit random generator, which needs a 64-bit unsigned long (Linux and macOS 64-bit).
- A CREXX source tree that provides the public header rxpa/crexxpa.h, and the generated header crexx_version.h. The plugin must be built against headers of the same CREXX version as the virtual machine that loads it.
- The CREXX driver (the crexx binary) that compiles and runs Level B programs. Rxpandas, rxscipy and rxnumpy need no installation into the CREXX bin directory.
- For the tests only: CPython with numpy, scipy and pandas, which produce the expected outputs.

Build

Variable	Meaning	Default
CREXX_ROOT	CREXX source tree providing rxpa/crexxpa.h	/home/claude/src/crexx
CREXX_GENERATED	directory with crexx_version.h	$CREXX_ROOT/cmake-build-release/generated
CREXX_VM	the crexx compiler driver that will run the code	$CREXX_ROOT/cmake-build-release/bin/crexx
RXLIBS_PLUGIN_DIR	output directory of the plugins	rxlibs/build/plugins
RXLIBS_LIB_DIR	directory of the compiled Level B libraries (rxspalg.rxbin)	rxlibs/build/lib
RXLIBS_WORK	scratch directory used by run.sh	rxlibs/build/work
CC	C compiler	gcc

Figure 4. Environment variables of the scripts

```
export CREXX_ROOT=/path/to/crexx
export CREXX_GENERATED=$CREXX_ROOT/cmake-build-release/generated
export CREXX_VM=/path/to/bin/crexx
cd rxlibs
scripts/build.sh              # build/plugins/rxnumpy.rxplugin rxscipy.rxplugin
    rxpandas.rxplugin
scripts/buildlib.sh           # build/lib/rxspalg.rxbin  (needs the plugins)
scripts/build.sh rxscipy      # a single plugin
```

Figure 5. Building the plugins and the Level B library

build.sh compiles each plugin out of tree with cc -O2 -fPIC -shared -DBUILD_DLL -DPLUGIN_ID=name. The plugin identifier must equal the namespace. The CREXX tree is not modified.

Compile and run a program

A Level B program is compiled with the plugin directory on the include path (-i), so that the compiler can read the function declarations from the .rxplugin files, and run with every plugin loaded (-l):

```
crexx prog.crexx -i build/plugins -i build/lib \
      -l build/plugins/rxnumpy.rxplugin -l build/plugins/rxscipy.rxplugin \
      -l build/plugins/rxpandas.rxplugin -l build/lib/rxspalg.rxbin
```

Figure 6. Compiling and running by hand

scripts/run.sh does exactly this for every plugin it finds, in a scratch directory because the driver writes .rxbin and .rxas files next to the source. The listings in this manual call it rxrun.

```
options levelb floats_binary
import rxfnsb
import rxnumpy

main: procedure = .int
  a = rxnumpy..np_array("[[1.0, 2.0], [3.0, 4.0]]")
  say "a @ a  =" rxnumpy..np_tolist(rxnumpy..np_dot(a, a))
  say "det(a) =" rxnumpy..np_det(a)
  /* handles share their data */
  g = a
  call rxnumpy..np_set(g, 0, 9.0)
  say "after np_set on the copy g:" rxnumpy..np_tolist(a)
  k = rxnumpy..np_copy(a)
  call rxnumpy..np_set(k, 0, 1.0)
  say "np_copy is independent:    " rxnumpy..np_tolist(a) rxnumpy..np_tolist(k)
  return 0

$ rxrun ex_hello.crexx
a @ a  = [[7.0, 10.0], [15.0, 22.0]]
det(a) = -2
after np_set on the copy g: [[9.0, 2.0], [3.0, 4.0]]
np_copy is independent:     [[9.0, 2.0], [3.0, 4.0]] [[1.0, 2.0], [3.0, 4.0]]
```

Figure 7. A first program

The program has a procedure main with the result type .int. Code that follows helper procedures outside main is not run (Chapter 10). The statement options levelb floats_binary selects Level B with binary floating point, which is required for floating point arguments. The import of rxfnsb provides the Level B builtin functions such as time().

Running the tests

```
scripts/runtests.sh
PASS t_basic
PASS t_fft_random
...
```

Figure 8. Test runner

runtests.sh builds the Level B library, regenerates the *.expected files from CPython with numpy, scipy and pandas through the gen_*.py scripts, runs every rx*/tests/t_*.crexx and compares. Appendix A lists the 9 test programs.

Chapter 3. The Handle Model and Error Signalling

Handles

An array or a table is a CREXX .binary value that carries a native payload: a small record with a magic number and a pointer to a reference counted C object. The type name stored in the payload ('crexx.rxnumpy.array.v1' or 'crexx.rxpandas.frame.v1') is checked by every function, so passing a table where an array is expected raises a signal instead of crashing.

- Assigning a handle to another variable or passing it as an argument copies the handle, not the data: the object is shared and reference counted. It is freed when the last CREXX value that refers to it dies. There is no free call.
- All functions that produce an array return a new handle. The inputs are never changed, with these exceptions that modify in place: np_set, np_set2, np_boolean_set, np_boolean_set_arr, np_rng_shuffle, pd_add_col, pd_add_str_col, pd_to_datetime and pd_dt_part.
- np_copy makes a deep copy.
- A handle must not outlive the plugin that created it; the virtual machine keeps plugins loaded for the whole run.

```
options levelb floats_binary
import rxfnsb
import rxnumpy

main: procedure = .int
  a = rxnumpy..np_array("[[1.0, 2.0], [3.0, 4.0]]")
  say "a @ a  =" rxnumpy..np_tolist(rxnumpy..np_dot(a, a))
  say "det(a) =" rxnumpy..np_det(a)
  /* handles share their data */
  g = a
  call rxnumpy..np_set(g, 0, 9.0)
  say "after np_set on the copy g:" rxnumpy..np_tolist(a)
  k = rxnumpy..np_copy(a)
  call rxnumpy..np_set(k, 0, 1.0)
  say "np_copy is independent:    " rxnumpy..np_tolist(a) rxnumpy..np_tolist(k)
  return 0

$ rxrun ex_hello.crexx
a @ a  = [[7.0, 10.0], [15.0, 22.0]]
det(a) = -2
after np_set on the copy g: [[9.0, 2.0], [3.0, 4.0]]
np_copy is independent:     [[9.0, 2.0], [3.0, 4.0]] [[1.0, 2.0], [3.0, 4.0]]
```

Figure 9. Sharing and copying

Shapes, text and complex numbers

- Shapes are strings: "3,4" or "(3, 4)". reshape allows one -1.
- np_array parses a Python-like nested list literal. Text with only integers produces int64, otherwise float64. np_tolist prints the same notation, floats with the shortest text that reads back identically (Python repr).
- There is no complex type. FFT results and eigenvalues are real arrays of shape (n, 2) with the real and the imaginary part; np_ifft accepts shape (n) or (n, 2).
- Comparisons give masks, which are int64 arrays of 0 and 1.

Out parameters

Functions with several results (np_qr, np_eigh, np_svd, sp_pearsonr, the t tests, sp_chi2_contingency, and the solvers of rxspalg) take the extra results as expose arguments. Declare a variable of the right type first and pass it: q = rxnumpy..np_zeros("1") for a handle, r = 0.0 for a float, n = 0 for an integer. The function returns 0, or its main result.

Errors

A failing call raises a CREXX signal through RETURNSIGNAL. INVALID_ARGUMENTS is raised for a bad handle, a shape mismatch, a singular matrix, an unknown column or distribution and similar; OUT_OF_RANGE for an index, row or column that does not exist; FAILURE for an allocation or handle publication error. The text from the C library is available as problem.message().

```
options levelb floats_binary
import rxfnsb
import rxnumpy
import rxpandas

main: procedure = .int
  do
    z = rxnumpy..np_inv(rxnumpy..np_array("[[1.0, 2.0], [2.0, 4.0]]"))
    say "not reached"
    on signal invalid_arguments as problem
    say "caught" problem.name() ":" problem.message()
  end
  do
    z = rxnumpy..np_get(rxnumpy..np_array("[1.0, 2.0]"), 5)
    say "not reached"
    on signal out_of_range as problem
    say "caught" problem.name() ":" problem.message()
  end
  do
    df = rxpandas..pd_parse_csv("a,b" || "0A"x || "1,2")
    z = rxpandas..pd_col(df, "nosuch")
    say "not reached"
    on signal invalid_arguments as problem
    say "caught" problem.name() ":" problem.message()
  end
  return 0

$ rxrun ex_errors.crexx
caught INVALID_ARGUMENTS : Singular matrix
caught OUT_OF_RANGE : index out of range
caught INVALID_ARGUMENTS : no such column
```

Figure 10. Catching the signals of the plugins

Memory and threads

The core allocates with malloc. Every core function returns either a new object or NULL with a message in a per-library static buffer (rxnp_error, rxsp_error, rxdf_error). The plugins are declared process reentrant, but the error buffers and the global random generators are not thread safe.

Chapter 4. Reference: rxnumpy

The functions of the plugin rxnumpy, namespace rxnumpy, are in the table below; each call is written rxnumpy..np_name(...). There are 141 functions. Every entry shows the arguments, the type of the result, a description, and an example with the output of really running it. The examples use these predefined values:

Name	Value
a	float matrix [[1.0, 2.0], [3.0, 4.0]]
m	float matrix [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]
v, w	float vectors [3.0, 1.0, 2.0] and [1.0, 2.0, 3.0]
iv	int64 vector [3, 1, 2]
mk	mask np_cmps(v, 1.5, ">") = [1, 0, 1]
spd	symmetric positive definite matrix [[4.0, 2.0], [2.0, 3.0]]
df, df2	tables df: city = Zurich, Basel, Zurich, Bern and sales = 10.5, 20.0, 30.25, 5.0; df2: city = Zurich, Bern, Geneva and pop = 420.0, 135.0, 203.0

Figure 11. Values predefined in the examples

Creating arrays

Call	Description	Example and result

`np_array(`
`  text = .string`
`)`
`  = .binary`
	Build an array from a Python-like nested list literal; all-integer text gives int64, otherwise float64.	`rxnumpy..np_array("[[1, 2], [3.5, 4]]")`
		`=> [[1.0, 2.0], [3.5, 4.0]]`

`np_zeros(`
`  shape = .string`
`)`
`  = .binary`
	Float64 array of zeros of the given shape string.	`rxnumpy..np_zeros("2,3")`
		`=> [[0.0, 0.0, 0.0], [0.0, 0.0, 0.0]]`

`np_ones(`
`  shape = .string`
`)`
`  = .binary`
	Float64 array of ones.	`rxnumpy..np_ones("(2, 2)")`
		`=> [[1.0, 1.0], [1.0, 1.0]]`

`np_full(`
`  shape = .string,`
`  value = .float`
`)`
`  = .binary`
	Float64 array filled with a value.	`rxnumpy..np_full("3", 7.5)`
		`=> [7.5, 7.5, 7.5]`

`np_zeros_int(`
`  shape = .string`
`)`
`  = .binary`
	Int64 array of zeros.	`rxnumpy..np_zeros_int("3")`
		`=> [0, 0, 0]`

`np_arange(`
`  start = .int,`
`  stop = .int,`
`  step = .int`
`)`
`  = .binary`
	Integer range start, start+step, ... below stop (int64).	`rxnumpy..np_arange(2, 11, 3)`
		`=> [2, 5, 8]`

`np_arangef(`
`  start = .float,`
`  stop = .float,`
`  step = .float`
`)`
`  = .binary`
	Floating point range (float64).	`rxnumpy..np_arangef(0.0, 1.0, 0.25)`
		`=> [0.0, 0.25, 0.5, 0.75]`

`np_linspace(`
`  start = .float,`
`  stop = .float,`
`  num = .int`
`)`
`  = .binary`
	num evenly spaced values from start to stop inclusive.	`rxnumpy..np_linspace(0.0, 1.0, 5)`
		`=> [0.0, 0.25, 0.5, 0.75, 1.0]`

`np_eye(`
`  n = .int`
`)`
`  = .binary`
	Identity matrix n x n.	`rxnumpy..np_eye(3)`
		`=> [[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 1.0]]`

`np_scalar(`
`  value = .float`
`)`
`  = .binary`
	0-d array holding one value.	`rxnumpy..np_scalar(2.5)`
		`=> 2.5`

Call	Description	Example and result

`np_copy(`
`  a = .binary`
`)`
`  = .binary`
	Deep copy (handles otherwise share their data).	`rxnumpy..np_copy(a)`
		`=> [[1.0, 2.0], [3.0, 4.0]]`

`np_asint(`
`  a = .binary`
`)`
`  = .binary`
	Convert to int64 (truncation).	`rxnumpy..np_asint(rxnumpy..np_ar`
		`ray("[1.7, -2.5]"))`
		`=> [1, -2]`

`np_asfloat(`
`  a = .binary`
`)`
`  = .binary`
	Convert to float64.	`rxnumpy..np_asfloat(iv)`
		`=> [3.0, 1.0, 2.0]`

Figure 12. Creating arrays

Properties and element access

Call	Description	Example and result

`np_ndim(`
`  a = .binary`
`)`
`  = .int`
	Number of dimensions.	`rxnumpy..np_ndim(m)`
		`=> 2`

`np_size(`
`  a = .binary`
`)`
`  = .int`
	Total number of elements.	`rxnumpy..np_size(m)`
		`=> 6`

`np_dim(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .int`
	Length of one axis.	`rxnumpy..np_dim(m, 1)`
		`=> 3`

`np_isint(`
`  a = .binary`
`)`
`  = .int`
	1 when the dtype is int64, else 0.	`rxnumpy..np_isint(iv)`
		`=> 1`

`np_shape(`
`  a = .binary`
`)`
`  = .string`
	Shape as the string "(2, 3)".	`rxnumpy..np_shape(m)`
		`=> 2,3`

`np_get(`
`  a = .binary,`
`  index = .int`
`)`
`  = .float`
	Element at a flat index (negative counts from the end) as float.	`rxnumpy..np_get(v, -1)`
		`=> 2`

`np_geti(`
`  a = .binary,`
`  index = .int`
`)`
`  = .int`
	Element at a flat index as int.	`rxnumpy..np_geti(iv, 0)`
		`=> 3`

`np_get2(`
`  a = .binary,`
`  i = .int,`
`  j = .int`
`)`
`  = .float`
	Element [i, j] of a matrix as float.	`rxnumpy..np_get2(a, 1, 0)`
		`=> 3`

`np_set(`
`  a = .binary,`
`  index = .int,`
`  value = .float`
`)`
`  = .int`
	Set the element at a flat index in place (all handles sharing the array see it); returns 0.	`rxnumpy..np_set(rxnumpy..np_copy`
		`(v), 0, 9.0)`
		`=> 0`

`np_set2(`
`  a = .binary,`
`  i = .int,`
`  j = .int,`
`  value = .float`
`)`
`  = .int`
	Set element [i, j] in place; returns 0.	`rxnumpy..np_set2(rxnumpy..np_cop`
		`y(a), 0, 1, 9.0)`
		`=> 0`

`np_tolist(`
`  a = .binary`
`)`
`  = .string`
	Text of the array as a Python nested list (floats in Python repr).	`rxnumpy..np_tolist(a)`
		`=> [[1.0, 2.0], [3.0, 4.0]]`

Figure 13. Properties and element access

Shape manipulation

Call	Description	Example and result

`np_reshape(`
`  a = .binary,`
`  shape = .string`
`)`
`  = .binary`
	New shape (one -1 allowed).	`rxnumpy..np_reshape(m, "3,-1")`
		`=> [[1.0, 2.0], [3.0, 4.0],`
		`[5.0, 6.0]]`

`np_transpose(`
`  a = .binary`
`)`
`  = .binary`
	Reverse the axes.	`rxnumpy..np_transpose(m)`
		`=> [[1.0, 4.0], [2.0, 5.0],`
		`[3.0, 6.0]]`

`np_flatten(`
`  a = .binary`
`)`
`  = .binary`
	1-d copy in C order.	`rxnumpy..np_flatten(a)`
		`=> [1.0, 2.0, 3.0, 4.0]`

`np_slice(`
`  a = .binary,`
`  axis = .int,`
`  start = .int,`
`  stop = .int,`
`  step = .int`
`)`
`  = .binary`
	Slice one axis, a[start:stop:step] along axis.	`rxnumpy..np_slice(m, 1, 0, 3, 2)`
		`=> [[1.0, 3.0], [4.0, 6.0]]`

`np_concat(`
`  a = .binary,`
`  b = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Concatenate along an axis.	`rxnumpy..np_concat(a, a, 0)`
		`=> [[1.0, 2.0], [3.0, 4.0],`
		`[1.0, 2.0], [3.0, 4.0]]`

`np_concatenate(`
`  a = .binary,`
`  b = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Concatenate along an axis (alias of np_concat for n-d inputs).	`rxnumpy..np_concatenate(m, m, 1)`
		`=> [[1.0, 2.0, 3.0, 1.0, 2.0,`
		`3.0], [4.0, 5.0, 6.0, 4.0, 5.0,`
		`6.0]]`

`np_stack(`
`  a = .binary,`
`  b = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Stack along a new axis.	`rxnumpy..np_stack(w, v, 0)`
		`=> [[1.0, 2.0, 3.0], [3.0, 1.0,`
		`2.0]]`

`np_vstack(`
`  a = .binary,`
`  b = .binary`
`)`
`  = .binary`
	Stack vertically.	`rxnumpy..np_vstack(w, v)`
		`=> [[1.0, 2.0, 3.0], [3.0, 1.0,`
		`2.0]]`

`np_hstack(`
`  a = .binary,`
`  b = .binary`
`)`
`  = .binary`
	Stack horizontally.	`rxnumpy..np_hstack(w, v)`
		`=> [1.0, 2.0, 3.0, 3.0, 1.0,`
		`2.0]`

Figure 14. Shape manipulation

Element-wise arithmetic

Call	Description	Example and result

`np_add(`
`  a = .binary,`
`  b = .binary`
`)`
`  = .binary`
	Element-wise a + b with broadcasting.	`rxnumpy..np_add(a,`
		`rxnumpy..np_array("[10.0,`
		`20.0]"))`
		`=> [[11.0, 22.0], [13.0, 24.0]]`

`np_sub(`
`  a = .binary,`
`  b = .binary`
`)`
`  = .binary`
	Element-wise a - b.	`rxnumpy..np_sub(a, a)`
		`=> [[0.0, 0.0], [0.0, 0.0]]`

Call	Description	Example and result
np_mul(a = .binary, b = .binary) = .binary	Element-wise a * b.	rxnumpy..np_mul(a, a) => [[1.0, 4.0], [9.0, 16.0]]
np_div(a = .binary, b = .binary) = .binary	Element-wise true division.	rxnumpy..np_div(a, rxnumpy..np_array("[2.0, 4.0]")) => [[0.5, 0.5], [1.5, 1.0]]
np_pow(a = .binary, b = .binary) = .binary	Element-wise power.	rxnumpy..np_pow(w, rxnumpy..np_array("[2.0, 2.0, 2.0]")) => [1.0, 4.0, 9.0]
np_mod(a = .binary, b = .binary) = .binary	Element-wise remainder with the sign of the divisor (like numpy).	rxnumpy..np_mod(rxnumpy..np_arra y("[-7.0, 7.0]"), rxnumpy..np_array("[3.0, - 3.0]")) => [2.0, -2.0]
np_floordiv(a = .binary, b = .binary) = .binary	Element-wise floor division.	rxnumpy..np_floordiv(rxnumpy..np _array("[-7.0, 7.0]"), rxnumpy..np_array("[2.0, 2.0]")) => [-4.0, 3.0]
np_maximum(a = .binary, b = .binary) = .binary	Element-wise maximum.	rxnumpy..np_maximum(v, w) => [3.0, 2.0, 3.0]
np_minimum(a = .binary, b = .binary) = .binary	Element-wise minimum.	rxnumpy..np_minimum(v, w) => [1.0, 1.0, 2.0]
np_arctan2(a = .binary, b = .binary) = .binary	Element-wise arctan(a / b) with quadrant.	rxnumpy..np_arctan2(rxnumpy..np_ array("[1.0, 1.0]"), rxnumpy..np_array("[1.0, - 1.0]")) => [0.7853981633974483, 2.356194490192345]
np_adds(a = .binary, s = .float) = .binary	Add a scalar to every element.	rxnumpy..np_adds(v, 0.5) => [3.5, 1.5, 2.5]
np_subs(a = .binary, s = .float) = .binary	Subtract a scalar.	rxnumpy..np_subs(v, 1.0) => [2.0, 0.0, 1.0]
np_muls(a = .binary, s = .float) = .binary	Multiply by a scalar.	rxnumpy..np_muls(v, 2.0) => [6.0, 2.0, 4.0]
np_divs(a = .binary, s = .float) = .binary	Divide by a scalar.	rxnumpy..np_divs(v, 2.0) => [1.5, 0.5, 1.0]
np_pows(a = .binary, s = .float) = .binary	Raise every element to a scalar power.	rxnumpy..np_pows(v, 2.0) => [9.0, 1.0, 4.0]
np_mods(a = .binary, s = .float) = .binary	Remainder by a scalar.	rxnumpy..np_mods(v, 2.0) => [1.0, 1.0, 0.0]

Call	Description	Example and result
np_neg(a = .binary) = .binary	Negation.	rxnumpy..np_neg(v) => [-3.0, -1.0, -2.0]
np_abs(a = .binary) = .binary	Absolute value.	rxnumpy..np_abs(rxnumpy..np_array("[-1.5, 2.0]")) => [1.5, 2.0]
np_clip(a = .binary, lo = .float, hi = .float) = .binary	Limit values to [lo, hi].	rxnumpy..np_clip(v, 1.5, 2.5) => [2.5, 1.5, 2.0]
np_sign(a = .binary) = .binary	-1, 0 or 1.	rxnumpy..np_sign(rxnumpy..np_array("[-2.5, 0.0, 3.0]")) => [-1.0, 0.0, 1.0]

Figure 15. Element-wise arithmetic

Element-wise mathematical functions

Call	Description	Example and result
np_sqrt(a = .binary) = .binary	Square root.	rxnumpy..np_sqrt(w) => [1.0, 1.4142135623730951, 1.7320508075688772]
np_exp(a = .binary) = .binary	e to the power x.	rxnumpy..np_exp(rxnumpy..np_array("[0.0, 1.0]")) => [1.0, 2.718281828459045]
np_log(a = .binary) = .binary	Natural logarithm.	rxnumpy..np_log(rxnumpy..np_array("[1.0, 2.718281828459045]")) => [0.0, 1.0]
np_sin(a = .binary) = .binary	Sine.	rxnumpy..np_sin(rxnumpy..np_array("[0.0, 1.5707963267948966]")) => [0.0, 1.0]
np_cos(a = .binary) = .binary	Cosine.	rxnumpy..np_cos(rxnumpy..np_array("[0.0, 3.141592653589793]")) => [1.0, -1.0]
np_tan(a = .binary) = .binary	Tangent.	rxnumpy..np_tan(rxnumpy..np_array("[0.0, 0.7853981633974483]")) => [0.0, 0.9999999999999999]
np_tanh(a = .binary) = .binary	Hyperbolic tangent.	rxnumpy..np_tanh(rxnumpy..np_array("[0.0, 1.0]")) => [0.0, 0.7615941559557649]
np_floor(a = .binary) = .binary	Round down.	rxnumpy..np_floor(rxnumpy..np_array("[1.5, -1.5]")) => [1.0, -2.0]
np_ceil(a = .binary) = .binary	Round up.	rxnumpy..np_ceil(rxnumpy..np_array("[1.5, -1.5]")) => [2.0, -1.0]
np_square(a = .binary) = .binary	x * x.	rxnumpy..np_square(v) => [9.0, 1.0, 4.0]
np_round(a = .binary) = .binary	Round half to even to integers (numpy round).	rxnumpy..np_round(rxnumpy..np_array("[0.5, 1.5, 2.5, 2.567]")) => [0.0, 2.0, 2.0, 3.0]
np_log10(a = .binary) = .binary	Base-10 logarithm.	rxnumpy..np_log10(rxnumpy..np_array("[1.0, 1000.0]")) => [0.0, 3.0]

Call	Description	Example and result
np_log2(a = .binary) = .binary	Base-2 logarithm.	rxnumpy..np_log2(rxnumpy..np_array("[1.0, 8.0]")) => [0.0, 3.0]
np_arctan(a = .binary) = .binary	Arc tangent.	rxnumpy..np_arctan(rxnumpy..np_array("[0.0, 1.0]")) => [0.0, 0.7853981633974483]

Figure 16. Element-wise mathematical functions

Reductions

Call	Description	Example and result
np_sum(a = .binary) = .float	Sum of all elements (pairwise summation, bit for bit as numpy).	rxnumpy..np_sum(m) => 21
np_sumi(a = .binary) = .int	Sum of an int64 array as int.	rxnumpy..np_sumi(iv) => 6
np_mean(a = .binary) = .float	Arithmetic mean.	rxnumpy..np_mean(m) => 3.5
np_min(a = .binary) = .float	Smallest element.	rxnumpy..np_min(v) => 1
np_max(a = .binary) = .float	Largest element.	rxnumpy..np_max(v) => 3
np_prod(a = .binary) = .float	Product of all elements.	rxnumpy..np_prod(v) => 6
np_std(a = .binary) = .float	Standard deviation (ddof 0).	rxnumpy..np_std(v) => 0.816496580927726
np_var(a = .binary) = .float	Variance (ddof 0).	rxnumpy..np_var(v) => 0.666666666666667
np_median(a = .binary) = .float	Median.	rxnumpy..np_median(v) => 2
np_std_ddof(a = .binary, ddof = .int) = .float	Standard deviation with ddof.	rxnumpy..np_std_ddof(v, 1) => 1
np_var_ddof(a = .binary, ddof = .int) = .float	Variance with ddof.	rxnumpy..np_var_ddof(v, 1) => 1
np_argmin(a = .binary) = .int	Flat index of the smallest element.	rxnumpy..np_argmin(v) => 1
np_argmax(a = .binary) = .int	Flat index of the largest element.	rxnumpy..np_argmax(v) => 0

Call	Description	Example and result

`np_sum_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Sum along an axis.	`rxnumpy..np_sum_axis(m, 0)`
		`=> [5.0, 7.0, 9.0]`

`np_mean_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Mean along an axis.	`rxnumpy..np_mean_axis(m, 1)`
		`=> [2.0, 5.0]`

`np_min_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Minimum along an axis.	`rxnumpy..np_min_axis(m, 1)`
		`=> [1.0, 4.0]`

`np_max_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Maximum along an axis.	`rxnumpy..np_max_axis(m, 1)`
		`=> [3.0, 6.0]`

`np_prod_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Product along an axis.	`rxnumpy..np_prod_axis(m, 1)`
		`=> [6.0, 120.0]`

`np_std_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Standard deviation along an axis.	`rxnumpy..np_std_axis(m, 0)`
		`=> [1.5, 1.5, 1.5]`

`np_var_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Variance along an axis.	`rxnumpy..np_var_axis(m, 0)`
		`=> [2.25, 2.25, 2.25]`

`np_median_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Median along an axis.	`rxnumpy..np_median_axis(m, 1)`
		`=> [2.0, 5.0]`

`np_argmin_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Index of the minimum along an axis.	`rxnumpy..np_argmin_axis(m, 1)`
		`=> [0, 0]`

`np_argmax_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Index of the maximum along an axis.	`rxnumpy..np_argmax_axis(m, 1)`
		`=> [2, 2]`

Figure 17. Reductions

Cumulative functions and sorting

Call	Description	Example and result

`np_cumsum(`
`  a = .binary`
`)`
`  = .binary`
	Cumulative sum of the flattened array.	`rxnumpy..np_cumsum(v)`
		`=> [3.0, 4.0, 6.0]`

`np_cumprod(`
`  a = .binary`
`)`
`  = .binary`
	Cumulative product of the flattened array.	`rxnumpy..np_cumprod(v)`
		`=> [3.0, 3.0, 6.0]`

`np_cumsum_axis(`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Cumulative sum along an axis.	`rxnumpy..np_cumsum_axis(m, 1)`
		`=> [[1.0, 3.0, 6.0], [4.0, 9.0, 15.0]]`

Call	Description	Example and result
np_cumprod_axis(a = .binary, axis = .int) = .binary	Cumulative product along an axis.	rxnumpy..np_cumprod_axis(m, 1) => [[1.0, 2.0, 6.0], [4.0, 20.0, 120.0]]
np_sort(a = .binary) = .binary	Sort along the last axis.	rxnumpy..np_sort(v) => [1.0, 2.0, 3.0]
np_argsort(a = .binary) = .binary	Indices that sort the array.	rxnumpy..np_argsort(v) => [1, 2, 0]
np_unique(a = .binary) = .binary	Sorted unique values.	rxnumpy..np_unique(rxnumpy..np_array("[3, 1, 3, 2, 1]")) => [1, 2, 3]
np_unique_counts(a = .binary) = .binary	Unique values and their counts as a 2 x k array.	rxnumpy..np_unique_counts(rxnumpy..np_array("[3, 1, 3, 2, 1]")) => [2, 1, 2]

Figure 18. Cumulative functions and sorting

Linear algebra

Call	Description	Example and result
np_dot(a = .binary, b = .binary) = .binary	Dot product of vectors, matrix product, matrix-vector product.	rxnumpy..np_dot(a, a) => [[7.0, 10.0], [15.0, 22.0]]
np_matmul(a = .binary, b = .binary) = .binary	numpy.matmul, including n-d broadcasting of the leading axes.	rxnumpy..np_matmul(a, a) => [[7.0, 10.0], [15.0, 22.0]]
np_tensordot(a = .binary, b = .binary, naxes = .int) = .binary	tensordot over the last naxes axes of a and the first naxes of b.	rxnumpy..np_tensordot(a, a, 1) => [[7.0, 10.0], [15.0, 22.0]]
np_outer(a = .binary, b = .binary) = .binary	Outer product of two vectors.	rxnumpy..np_outer(w, w) => [[1.0, 2.0, 3.0], [2.0, 4.0, 6.0], [3.0, 6.0, 9.0]]
np_trace(a = .binary) = .float	Sum of the diagonal.	rxnumpy..np_trace(a) => 5
np_norm(a = .binary) = .float	Frobenius / 2-norm.	rxnumpy..np_norm(rxnumpy..np_array("[3.0, 4.0]")) => 5
np_solve(a = .binary, b = .binary) = .binary	Solve a x = b (LU with partial pivoting). Singular matrices raise INVALID_ARGUMENTS.	rxnumpy..np_solve(a, rxnumpy..np_array("[5.0, 6.0]")) => [-3.9999999999999987, 4.499999999999999]
np_inv(a = .binary) = .binary	Matrix inverse.	rxnumpy..np_inv(a) => [[-1.9999999999999996, 0.9999999999999998], [1.4999999999999998, -0.4999999999999999]]
np_det(a = .binary) = .float	Determinant.	rxnumpy..np_det(a) => -2

Call	Description	Example and result

`np_cholesky(`
`  a = .binary`
`)`
`  = .binary`
	Lower Cholesky factor of a positive definite matrix.	`rxnumpy..np_cholesky(spd)`
		`=> [[2.0, 0.0], [1.0, 1.4142135623730951]]`

`np_eigvals(`
`  a = .binary`
`)`
`  = .binary`
	Eigenvalues as an (n, 2) array of [re, im].	`rxnumpy..np_eigvals(spd)`
		`=> [[5.561552812808831, 0.0], [1.4384471871911697, 0.0]]`

`np_qr(`
`  a = .binary,`
`  expose q = .binary,`
`  expose r = .binary`
`)`
`  = .int`
	Reduced QR; q and r are out parameters, returns 0.	`q = rxnumpy..np_zeros("1")`
		`r = rxnumpy..np_zeros("1")`
		`ignore = rxnumpy..np_qr(a, q, r)`
		`say rxnumpy..np_tolist(rxnumpy..np_round(rxnumpy..np_muls(r, 1000.0)))`
		`=> [[-3162.0, -4427.0], [0.0, -632.0]]`

`np_eigh(`
`  a = .binary,`
`  expose w = .binary,`
`  expose v = .binary`
`)`
`  = .int`
	Symmetric eigen decomposition, eigenvalues ascending; w and v are out parameters.	`ew = rxnumpy..np_zeros("1")`
		`vv = rxnumpy..np_zeros("1")`
		`ignore = rxnumpy..np_eigh(spd, ew, vv)`
		`say rxnumpy..np_tolist(rxnumpy..np_round(rxnumpy..np_muls(ew, 1000.0)))`
		`=> [1438.0, 5562.0]`

`np_svd(`
`  a = .binary,`
`  expose u = .binary,`
`  expose s = .binary,`
`  expose vt = .binary`
`)`
`  = .int`
	Reduced SVD; u, s and vt are out parameters.	`u = rxnumpy..np_zeros("1")`
		`s = rxnumpy..np_zeros("1")`
		`vt = rxnumpy..np_zeros("1")`
		`ignore = rxnumpy..np_svd(spd, u, s, vt)`
		`say rxnumpy..np_tolist(rxnumpy..np_round(rxnumpy..np_muls(s, 1000.0)))`
		`=> [5562.0, 1438.0]`

Figure 19. Linear algebra

Fourier transform

Call	Description	Example and result

`np_fft(`
`  a = .binary`
`)`
`  = .binary`
	FFT of a real or (n,2) array; result shape (n,2) = [re, im] (radix 2 and Bluestein).	`rxnumpy..np_round(rxnumpy..np_fft(rxnumpy..np_array("[1.0, 2.0, 3.0, 4.0]")))`
		`=> [[10.0, 0.0], [-2.0, 2.0], [-2.0, 0.0], [-2.0, -2.0]]`

`np_ifft(`
`  a = .binary`
`)`
`  = .binary`
	Inverse FFT.	`rxnumpy..np_round(rxnumpy..np_ifft(rxnumpy..np_fft(rxnumpy..np_array("[1.0, 2.0, 3.0, 4.0]"))))`
		`=> [[1.0, 0.0], [2.0, 0.0], [3.0, 0.0], [4.0, 0.0]]`

Figure 20. Fourier transform

Legacy random numbers (MT19937)

Call	Description	Example and result

`np_rand_seed(`
`  seed = .int`
`)`
`  = .int`
	Seed the legacy MT19937 stream (np.random.seed).	`rxnumpy..np_rand_seed(42)`
		`=> 0`

`np_rand(`
`  shape = .string`
`)`
`  = .binary`
	Uniform [0, 1) array from the legacy stream (np.random.rand).	`ignore = rxnumpy..np_rand_seed(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rand("3"))`
		`=> [0.3745401188473625, 0.9507143064099162, 0.7319939418114051]`

Call	Description	Example and result

`np_randn(`
`  shape = .string`
`)`
`  = .binary`
	Standard normal array from the legacy stream (np.random.randn).	`ignore =`
		`rxnumpy..np_rand_seed(42)`
		`say`
		`rxnumpy..np_tolist(rxnumpy..np_randn("3"))`
		`=> [0.4967141530112327, -0.13826430117118466, 0.6476885381006925]`

`np_randint(`
`  low = .int,`
`  high = .int,`
`  shape = .string`
`)`
`  = .binary`
	Integers in [low, high) from the legacy stream.	`ignore =`
		`rxnumpy..np_rand_seed(42)`
		`say`
		`rxnumpy..np_tolist(rxnumpy..np_randint(0, 10, "5"))`
		`=> [6, 3, 7, 4, 6]`

`np_random()`
`  = .float`
	One uniform number from the legacy stream.	`ignore =`
		`rxnumpy..np_rand_seed(1)`
		`say rxnumpy..np_random()`
		`=> 0.417022004702574`

`np_gauss()`
`  = .float`
	One standard normal number from the legacy stream.	`ignore =`
		`rxnumpy..np_rand_seed(1)`
		`say rxnumpy..np_gauss()`
		`=> 1.62434536366324`

Figure 21. Legacy random numbers (MT19937)

Generator (default_rng, PCG64)

Call	Description	Example and result

`np_default_rng(`
`  seed = .int`
`)`
`  = .binary`
	numpy.random.default_rng(seed): SeedSequence + PCG64 Generator handle.	`g = rxnumpy..np_default_rng(7)`
		`say rxnumpy..np_rng_random1(g)`
		`=> 0.625095466604667`

`np_default_rng_arr(`
`  seed = .binary`
`)`
`  = .binary`
	default_rng seeded with an int64 array.	`g = rxnumpy..np_default_rng_arr(rxnumpy..np_array("[1, 2, 3]"))`
		`say rxnumpy..np_rng_random1(g)`
		`=> 0.670472262651663`

`np_rng_random(`
`  rng = .binary,`
`  shape = .string`
`)`
`  = .binary`
	Generator.random(shape).	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rng_random(g, "3"))`
		`=> [0.7739560485559633, 0.4388784397520523, 0.8585979199113825]`

`np_rng_random1(`
`  rng = .binary`
`)`
`  = .float`
	One Generator.random() value.	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_rng_random1(g)`
		`=> 0.773956048555963`

`np_rng_normal(`
`  rng = .binary,`
`  loc = .float,`
`  scale = .float,`
`  shape = .string`
`)`
`  = .binary`
	Generator.normal(loc, scale, shape) (ziggurat).	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rng_normal(g, 0.0, 1.0, "3"))`
		`=> [0.30471707975443135, -1.0399841062404955, 0.7504511958064572]`

`np_rng_normal1(`
`  rng = .binary`
`)`
`  = .float`
	One standard normal draw.	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_rng_normal1(g)`
		`=> 0.304717079754431`

`np_rng_exponential(`
`  rng = .binary,`
`  scale = .float,`
`  shape = .string`
`)`
`  = .binary`
	Generator.exponential(scale, shape).	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rng_exponential(g, 2.0, "3"))`
		`=> [4.808417207931989, 4.672379311648907, 4.76952199974851]`

`np_rng_integers(`
`  rng = .binary,`
`  low = .int,`
`  high = .int,`
`  shape = .string`
`)`
`  = .binary`
	Generator.integers(low, high) with exclusive high.	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rng_integers(g, 0, 10, "5"))`
		`=> [0, 7, 6, 4, 4]`

Call	Description	Example and result

`np_rng_integer1(`
`  rng = .binary,`
`  low = .int,`
`  high = .int`
`)`
`  = .int`
	One integer in [low, high).	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_rng_integer1(g, 0, 100)`
		`=> 8`

`np_rng_choice(`
`  rng = .binary,`
`  a = .binary,`
`  size = .int,`
`  replace = .int`
`)`
`  = .binary`
	Generator.choice(a, size, replace) for 1-d a, uniform weights.	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rng_choice(g, rxnumpy..np_arange(0, 10, 1), 3, 0))`
		`=> [9, 0, 6]`

`np_rng_shuffle(`
`  rng = .binary,`
`  a = .binary`
`)`
`  = .int`
	Shuffle a 1-d array in place; returns 0.	`g = rxnumpy..np_default_rng(42)`
		`x = rxnumpy..np_arange(0, 6, 1)`
		`ignore = rxnumpy..np_rng_shuffle(g, x)`
		`say rxnumpy..np_tolist(x)`
		`=> [3, 2, 5, 4, 1, 0]`

`np_rng_permutation(`
`  rng = .binary,`
`  n = .int`
`)`
`  = .binary`
	Random permutation of 0..n-1.	`g = rxnumpy..np_default_rng(42)`
		`say rxnumpy..np_tolist(rxnumpy..np_rng_permutation(g, 6))`
		`=> [3, 2, 5, 4, 1, 0]`

Figure 22. Generator (default_rng, PCG64)

Comparison, masks and indexing

Call	Description	Example and result

`np_cmp(`
`  a = .binary,`
`  b = .binary,`
`  op = .string`
`)`
`  = .binary`
	Element-wise comparison (> < >= <= == !=) giving an int64 0/1 mask.	`rxnumpy..np_cmp(v, w, "<")`
		`=> [0, 1, 1]`

`np_cmps(`
`  a = .binary,`
`  s = .float,`
`  op = .string`
`)`
`  = .binary`
	Comparison with a scalar.	`rxnumpy..np_cmps(v, 1.5, ">")`
		`=> [1, 0, 1]`

`np_take(`
`  a = .binary,`
`  idx = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Take indices along an axis.	`rxnumpy..np_take(m, rxnumpy..np_array("[2, 0]"), 1)`
		`=> [[3.0, 1.0], [6.0, 4.0]]`

`np_take_flat(`
`  a = .binary,`
`  idx = .binary`
`)`
`  = .binary`
	Take indices from the flattened array.	`rxnumpy..np_take_flat(v, rxnumpy..np_array("[2, 0]"))`
		`=> [2.0, 3.0]`

`np_where(`
`  cond = .binary,`
`  a = .binary,`
`  b = .binary`
`)`
`  = .binary`
	where(cond, a, b) with broadcasting.	`rxnumpy..np_where(mk, v, w)`
		`=> [3.0, 2.0, 2.0]`

`np_compress(`
`  mask = .binary,`
`  a = .binary,`
`  axis = .int`
`)`
`  = .binary`
	Keep entries of an axis where the mask is 1.	`rxnumpy..np_compress(rxnumpy..np_array("[1, 0, 1]"), m, 1)`
		`=> [[1.0, 3.0], [4.0, 6.0]]`

`np_boolean_get(`
`  a = .binary,`
`  mask = .binary`
`)`
`  = .binary`
	a[mask]: elements where the mask is 1, as a 1-d array.	`rxnumpy..np_boolean_get(v, mk)`
		`=> [3.0, 2.0]`

Call	Description	Example and result

`np_boolean_set(`
`  a = .binary,`
`  mask = .binary,`
`  value = .float`
`)`
`  = .int`
	a[mask] = value in place; returns 0.	`x = rxnumpy..np_copy(v)`
		`ignore =`
		`rxnumpy..np_boolean_set(x, mk,`
		`0.0)`
		`say rxnumpy..np_tolist(x)`
		`=> [0.0, 1.0, 0.0]`

`np_boolean_set_arr(`
`  a = .binary,`
`  mask = .binary,`
`  values = .binary`
`)`
`  = .int`
	a[mask] = values (one value per True) in place; returns 0.	`x = rxnumpy..np_copy(v)`
		`ignore =`
		`rxnumpy..np_boolean_set_arr(x,`
		`mk, rxnumpy..np_array("[10.0]"))`
		`say rxnumpy..np_tolist(x)`
		`=> [10.0, 1.0, 10.0]`

Figure 23. Comparison, masks and indexing

Chapter 5. Reference: rxscipy

The functions of rxscipy, namespace rxscipy. There are 51 functions. Scalars are .float or .int, vectors and matrices are rxnumpy handles. The examples use the same predefined values as the previous chapter.

Distributions

Twelve distributions are available through one set of functions that take the distribution name as the first argument: sp_pdf, sp_pmf, sp_cdf, sp_sf, sp_ppf, sp_mean, sp_var and sp_rvs. The parameters are always (a, b, loc, scale), the shape parameters of scipy.stats first, then location and scale; parameters a distribution does not use are ignored. Discrete distributions ignore scale and use loc as a shift.

dist	a	b	scipy.stats equivalent
norm	-	-	norm(loc, scale)
t	df	-	t(df, loc, scale)
chi2	df	-	chi2(df, loc, scale)
f	dfn	dfd	f(dfn, dfd, loc, scale)
gamma	a	-	gamma(a, loc, scale)
beta	a	b	beta(a, b, loc, scale)
expon	-	-	expon(loc, scale)
uniform	-	-	uniform(loc, scale)
lognorm	s	-	lognorm(s, loc, scale)
binom	n	p	binom(n, p, loc)
poisson	mu	-	poisson(mu, loc)
geom	p	-	geom(p, loc)

Figure 24. Distributions and their parameters

pdf, cdf, sf and ppf agree with scipy to about 1e-13 relative (Appendix C). sf and the upper tails are computed from the complementary functions, so small tail probabilities keep their precision. ppf of the continuous distributions without closed form is a safeguarded Newton iteration on the smaller tail. For the discrete distributions ppf is the smallest integer k with cdf(k) >= q.

Random variates

sp_seed(s) seeds the legacy MT19937 stream of rxscipy, and sp_rvs(dist, n, a, b, loc, scale) draws from it with the algorithms of numpy's legacy RandomState (Marsaglia-Tsang gamma, the polar normal, Johnk beta, and the others). After sp_seed(s) the numbers equal those of np.random.seed(s) followed by scipy.stats.dist.rvs(size=n). This is bit-exact for norm, t, chi2, f, gamma, beta, expon, uniform, lognorm and geom, for binom when n*min(p, 1-p) is at most 30 and for poisson when mu is below 10. For larger parameters numpy uses the BTPE and PTRS rejection algorithms; sp_rvs then uses inversion of the cdf with one uniform number: the sample is correct in distribution but not identical to numpy's. The stream is separate from the stream of np_rand_seed (each plugin library has its own state): seed both when the two are used together.

Scalar mathematics

Call	Description	Example and result

`sp_sqrt(`
`  x = .float`
`)`
`  = .float`
	Square root of a float.	`rxscipy..sp_sqrt(2.0)`
		`=> 1.4142135623731`

`sp_exp(`
`  x = .float`
`)`
`  = .float`
	e to the power x.	`rxscipy..sp_exp(1.0)`
		`=> 2.71828182845905`

`sp_log(`
`  x = .float`
`)`
`  = .float`
	Natural logarithm.	`rxscipy..sp_log(10.0)`
		`=> 2.30258509299405`

Call	Description	Example and result

`sp_fabs(`
`  x = .float`
`)`
`  = .float`
	Absolute value.	`rxscipy..sp_fabs(-2.5)`
		`=> 2.5`

`sp_sin(`
`  x = .float`
`)`
`  = .float`
	Sine.	`rxscipy..sp_sin(1.0)`
		`=> 0.841470984807896`

`sp_cos(`
`  x = .float`
`)`
`  = .float`
	Cosine.	`rxscipy..sp_cos(1.0)`
		`=> 0.540302305868140`

`sp_floor(`
`  x = .float`
`)`
`  = .float`
	Round down.	`rxscipy..sp_floor(-1.5)`
		`=> -2`

`sp_pow(`
`  x = .float,`
`  y = .float`
`)`
`  = .float`
	x to the power y.	`rxscipy..sp_pow(2.0, 10.0)`
		`=> 1024`

`sp_isfinite(`
`  x = .float`
`)`
`  = .int`
	1 when x is neither infinite nor NaN.	`rxscipy..sp_isfinite(1.0)`
		`=> 1`

Figure 25. Scalar mathematics

Special functions

Call	Description	Example and result

`sp_erf(`
`  x = .float`
`)`
`  = .float`
	Error function (relative error about 1e-14).	`rxscipy..sp_erf(0.5)`
		`=> 0.520499877813047`

`sp_erfc(`
`  x = .float`
`)`
`  = .float`
	Complementary error function, accurate in the tail.	`rxscipy..sp_erfc(3.0)`
		`=> 0.0000220904969985855`

`sp_erfinv(`
`  y = .float`
`)`
`  = .float`
	Inverse error function.	`rxscipy..sp_erfinv(0.5)`
		`=> 0.47693627620447`

`sp_gamma(`
`  x = .float`
`)`
`  = .float`
	Gamma function (exact for integers up to 171).	`rxscipy..sp_gamma(5.5)`
		`=> 52.3427777845534`

`sp_lgamma(`
`  x = .float`
`)`
`  = .float`
	Natural log of |gamma(x)|.	`rxscipy..sp_lgamma(10.5)`
		`=> 13.9406252194038`

`sp_beta(`
`  a = .float,`
`  b = .float`
`)`
`  = .float`
	Beta function B(a, b).	`rxscipy..sp_beta(2.0, 3.0)`
		`=> 0.0833333333333333`

`sp_betainc(`
`  a = .float,`
`  b = .float,`
`  x = .float`
`)`
`  = .float`
	Regularised incomplete beta function I_x(a, b).	`rxscipy..sp_betainc(2.0, 3.0, 0.4)`
		`=> 0.5248`

`sp_gammainc(`
`  a = .float,`
`  x = .float`
`)`
`  = .float`
	Regularised lower incomplete gamma P(a, x).	`rxscipy..sp_gammainc(3.0, 2.0)`
		`=> 0.323323583816936`

`sp_gammaincc(`
`  a = .float,`
`  x = .float`
`)`
`  = .float`
	Regularised upper incomplete gamma Q(a, x).	`rxscipy..sp_gammaincc(3.0, 2.0)`
		`=> 0.676676416183064`

Figure 26. Special functions

Normal and Student t shortcuts

Call	Description	Example and result

`sp_norm_pdf(`
`  x = .float`
`)`
`  = .float`
	Standard normal density.	`rxscipy..sp_norm_pdf(0.0)`
		`=> 0.398942280401433`

`sp_norm_cdf(`
`  x = .float`
`)`
`  = .float`
	Standard normal distribution function.	`rxscipy..sp_norm_cdf(1.96)`
		`=> 0.975002104851779`

`sp_norm_ppf(`
`  p = .float`
`)`
`  = .float`
	Standard normal quantile.	`rxscipy..sp_norm_ppf(0.975)`
		`=> 1.95996398454006`

`sp_t_cdf(`
`  t = .float,`
`  df = .float`
`)`
`  = .float`
	Student t distribution function (df degrees of freedom).	`rxscipy..sp_t_cdf(2.0, 5.0)`
		`=> 0.94903026058507`

Figure 27. Normal and Student t shortcuts

Distributions: pdf, pmf, cdf, sf, ppf, mean, var

Call	Description	Example and result

`sp_pdf(`
`  dist = .string,`
`  x = .float,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Density of distribution dist at x. dist: norm t chi2 f gamma beta expon uniform lognorm; shape parameters a, b (t:df; chi2:df; f:dfn,dfd; gamma:a; beta:a,b; lognorm:s), then loc and scale. Unused parameters are ignored.	`rxscipy..sp_pdf("gamma", 2.0, 2.5, 0.0, 0.0, 1.5)`
		`=> 0.203526674668666`

`sp_pmf(`
`  dist = .string,`
`  x = .float,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Probability mass of binom (a=n, b=p), poisson (a=mu) or geom (a=p) at integer x; loc shifts.	`rxscipy..sp_pmf("binom", 3.0, 10.0, 0.3, 0.0, 1.0)`
		`=> 0.266827932`

`sp_cdf(`
`  dist = .string,`
`  x = .float,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Distribution function.	`rxscipy..sp_cdf("t", 2.0, 7.0, 0.0, 0.0, 1.0)`
		`=> 0.957190335718512`

`sp_sf(`
`  dist = .string,`
`  x = .float,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Survival function 1 - cdf, accurate in the upper tail.	`rxscipy..sp_sf("chi2", 12.0, 4.0, 0.0, 0.0, 1.0)`
		`=> 0.0173512652366645`

`sp_ppf(`
`  dist = .string,`
`  q = .float,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Quantile (inverse cdf); for discrete distributions the smallest k with cdf(k) >= q.	`rxscipy..sp_ppf("f", 0.95, 5.0, 8.0, 0.0, 1.0)`
		`=> 3.68749866634002`

Call	Description	Example and result

`sp_mean(`
`  dist = .string,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Mean of the distribution (NaN when it does not exist).	`rxscipy..sp_mean("gamma", 2.5, 0.0, 0.0, 1.5)`
		`=> 3.75`

`sp_var(`
`  dist = .string,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .float`
	Variance of the distribution.	`rxscipy..sp_var("beta", 2.0, 3.0, 0.0, 1.0)`
		`=> 0.04`

Figure 28. Distributions: pdf, pmf, cdf, sf, ppf, mean, var

Random variates

Call	Description	Example and result

`sp_seed(`
`  seed = .int`
`)`
`  = .int`
	Seed the rxscipy legacy MT19937 stream used by sp_rvs (like np.random.seed).	`rxscipy..sp_seed(42)`
		`=> 0`

`sp_rvs(`
`  dist = .string,`
`  size = .int,`
`  a = .float,`
`  b = .float,`
`  loc = .float,`
`  scale = .float`
`)`
`  = .binary`
	size random variates; after sp_seed(s) the numbers equal np.random.seed(s); dist.rvs(size=n) of scipy (exceptions: Chapter 8). Discrete distributions give int64.	`ignore = rxscipy..sp_seed(42)`
		`say rxnumpy..np_tolist(rxscipy..sp_rvs("norm", 3, 0.0, 0.0, 0.0, 1.0))`
		`=> [0.4967141530112327, -0.13826430117118466, 0.6476885381006925]`

Figure 29. Random variates

Integration, interpolation and polynomials

Call	Description	Example and result

`sp_trapz(`
`  y = .binary,`
`  x = .binary`
`)`
`  = .float`
	Trapezoid rule for samples y at points x.	`rxscipy..sp_trapz(w, w)`
		`=> 4`

`sp_trapz_unit(`
`  y = .binary`
`)`
`  = .float`
	Trapezoid rule with unit spacing.	`rxscipy..sp_trapz_unit(v)`
		`=> 3.5`

`sp_simpson(`
`  y = .binary,`
`  x = .binary`
`)`
`  = .float`
	Simpson rule (odd number of samples).	`rxscipy..sp_simpson(w, w)`
		`=> 4`

`sp_interp(`
`  x = .binary,`
`  xp = .binary,`
`  fp = .binary`
`)`
`  = .binary`
	Piecewise linear interpolation (numpy.interp) at x from the table xp, fp.	`rxscipy..sp_interp(rxnumpy..np_array("[0.5, 1.5]"), w, v)`
		`=> [3.0, 2.0]`

`sp_cubic_spline(`
`  x = .binary,`
`  y = .binary,`
`  xnew = .binary,`
`  natural = .int,`
`  deriv = .int`
`)`
`  = .binary`
	CubicSpline through (x, y), evaluated at xnew; natural = 0 not-a-knot, 1 natural; deriv 0..2.	`rxscipy..sp_cubic_spline(rxnumpy..np_array("[0.0, 1.0, 2.0, 3.0]"), rxnumpy..np_array("[0.0, 1.0, 8.0, 27.0]"), rxnumpy..np_array("[1.5]"), 0, 0)`
		`=> [3.375]`

Call	Description	Example and result

`sp_polyfit(`
`  x = .binary,`
`  y = .binary,`
`  deg = .int`
`)`
`  = .binary`
	Least squares polynomial of degree deg, highest power first.	`rxnumpy..np_divs(rxnumpy..np_round(rxnumpy..np_muls(rxscipy..sp_polyfit(rxnumpy..np_array("[0.0, 1.0, 2.0, 3.0]"), rxnumpy..np_array("[1.0, 3.0, 5.0, 7.0]"), 1), 1000.0)), 1000.0)`
		`=> [2.0, 1.0]`

`sp_polyval(`
`  c = .binary,`
`  x = .binary`
`)`
`  = .binary`
	Evaluate a polynomial (highest power first).	`rxscipy..sp_polyval(rxnumpy..np_array("[2.0, 1.0]"), w)`
		`=> [3.0, 5.0, 7.0]`

Figure 30. Integration, interpolation and polynomials

Descriptive statistics

Call	Description	Example and result

`sp_describe(`
`  x = .binary`
`)`
`  = .binary`
	Array [nobs, min, max, mean, variance (ddof 1), skewness, kurtosis (Fisher)].	`rxscipy..sp_describe(v)`
		`=> [3.0, 1.0, 3.0, 2.0, 1.0, 0.0, -1.5]`

`sp_skew(`
`  x = .binary`
`)`
`  = .float`
	Sample skewness (biased).	`rxscipy..sp_skew(rxnumpy..np_array("[1.0, 2.0, 2.0, 9.0]"))`
		`=> 1.09702544936387`

`sp_kurtosis(`
`  x = .binary`
`)`
`  = .float`
	Excess kurtosis (Fisher, biased).	`rxscipy..sp_kurtosis(rxnumpy..np_array("[1.0, 2.0, 2.0, 9.0]"))`
		`=> -0.70553242117787`

`sp_zscore(`
`  x = .binary,`
`  ddof = .int`
`)`
`  = .binary`
	(x - mean) / std with the given ddof.	`rxscipy..sp_zscore(w, 0)`
		`=> [-1.224744871391589, 0.0, 1.224744871391589]`

`sp_percentile(`
`  x = .binary,`
`  q = .float`
`)`
`  = .float`
	Percentile q in [0, 100] with linear interpolation.	`rxscipy..sp_percentile(rxnumpy..np_array("[1.0, 2.0, 3.0, 4.0]"), 25.0)`
		`=> 1.75`

`sp_correlate(`
`  a = .binary,`
`  v = .binary,`
`  mode = .string`
`)`
`  = .binary`
	numpy.correlate(a, v, mode) for mode valid, same or full.	`rxscipy..sp_correlate(w, rxnumpy..np_array("[0.0, 1.0, 0.5]"), "full")`
		`=> [0.5, 2.0, 3.5, 3.0, 0.0]`

Figure 31. Descriptive statistics

Statistical tests

Call	Description	Example and result

`sp_pearsonr(`
`  x = .binary,`
`  y = .binary,`
`  expose r = .float,`
`  expose p = .float`
`)`
`  = .int`
	Pearson correlation r and two-sided p-value (out parameters); returns 0.	`r = 0.0`
		`p = 0.0`
		`ignore = rxscipy..sp_pearsonr(w, rxnumpy..np_array("[1.0, 3.0, 2.0]"), r, p)`
		`say r p`
		`=> 0.5 0.666666666666666`

`sp_spearmanr(`
`  x = .binary,`
`  y = .binary,`
`  expose rho = .float,`
`  expose p = .float`
`)`
`  = .int`
	Spearman rank correlation (ties get average ranks) and p-value; returns 0.	`r = 0.0`
		`p = 0.0`
		`ignore = rxscipy..sp_spearmanr(rxnumpy..np_array("[1.0, 2.0, 3.0, 4.0]"), rxnumpy..np_array("[1.0, 3.0, 2.0, 4.0]"), r, p)`
		`say r p`
		`=> 0.8 0.2`

Call	Description	Example and result

`sp_ttest_1samp(`
`  x = .binary,`
`  popmean = .float,`
`  expose t = .float,`
`  expose p = .float`
`)`
`  = .int`
	One-sample t test against popmean: t and p (out); returns 0.	`r = 0.0`
		`p = 0.0`
		`ignore =`
		`rxscipy..sp_ttest_1samp(v, 1.0, r, p)`
		`say r p`
		`=> 1.73205080756888`
		`0.225403330758517`

`sp_ttest_ind(`
`  a = .binary,`
`  b = .binary,`
`  expose t = .float,`
`  expose p = .float`
`)`
`  = .int`
	Two-sample t test with equal variances: t and p (out); returns 0.	`r = 0.0`
		`p = 0.0`
		`ignore =`
		`rxscipy..sp_ttest_ind(v,`
		`rxnumpy..np_array("[5.0, 6.0, 4.0, 7.0]"), r, p)`
		`say r p`
		`=> -3.87298334620742`
		`0.0117248110039547`

`sp_ttest_welch(`
`  a = .binary,`
`  b = .binary,`
`  expose t = .float,`
`  expose p = .float,`
`  expose df = .float`
`)`
`  = .int`
	Welch t test (unequal variances): t, p and the Welch df (out); returns 0.	`r = 0.0`
		`p = 0.0`
		`d = 0.0`
		`ignore =`
		`rxscipy..sp_ttest_welch(v,`
		`rxnumpy..np_array("[5.0, 6.0, 4.0, 7.0]"), r, p, d)`
		`say r p d`
		`=> -4.04145188432738`
		`0.0100769433479888`
		`4.95918367346939`

`sp_chi2_contingency(`
`  table = .binary,`
`  correction = .int,`
`  expose chi2 = .float,`
`  expose p = .float,`
`  expose dof = .int,`
`  expose expected = .binary`
`)`
`  = .int`
	Chi-square test of independence for a 2-d table: statistic, p, dof and expected frequencies (out); correction = 1 applies Yates when dof is 1.	`c = 0.0`
		`p = 0.0`
		`n = 0`
		`e = rxnumpy..np_zeros("1")`
		`ignore =`
		`rxscipy..sp_chi2_contingency(rxnumpy..np_array("[[10, 20], [30, 25]]"), 1, c, p, n, e)`
		`say c p n rxnumpy..np_tolist(e)`
		`=> 2.7061553030303`
		`0.0999616438735336 1`
		`[[14.117647058823529,`
		`15.882352941176471],`
		`[25.88235294117647,`
		`29.11764705882353]]`

`sp_linregress(`
`  x = .binary,`
`  y = .binary`
`)`
`  = .binary`
	Linear regression: array [slope, intercept, rvalue, pvalue, stderr, intercept_stderr].	`rxnumpy..np_divs(rxnumpy..np_round(rxnumpy..np_muls(rxscipy..sp_linregress(rxnumpy..np_array("[1.0, 2.0, 3.0, 4.0]"),`
		`rxnumpy..np_array("[2.1, 3.9, 6.2, 7.8]")), 1000.0)), 1000.0)`
		`=> [1.94, 0.15, 0.998, 0.002, 0.091, 0.248]`

Figure 32. Statistical tests

Chapter 6. Reference: rxpandas

The functions of rxpandas, namespace rxpandas. There are 33 functions. A table is a handle with named columns, each either float64 or string, and an index (the default range index, or labels from pd_set_index). Filter, sort and head keep the original row positions as index labels, as pandas does. A merge leaves the string cells of unmatched rows empty and the numeric cells NaN. Datetimes are epoch seconds in a float column.

Creating and inspecting tables

Call	Description	Example and result
`pd_new()` `  = .binary`	Empty table.	`rxpandas..pd_nrows(rxpandas..pd_new())` `=> 0`

`pd_read_csv(`
`  path = .string`
`)`
`  = .binary`
	Read a CSV file (header row; numeric columns become float64, the rest string columns).	`t = rxpandas..pd_parse_csv("a,b" || "0A"x || "1,2")`
		`ignore = rxpandas..pd_to_csv(t, "/tmp/rxlibs_doc_example.csv")`
		`say rxpandas..pd_nrows(rxpandas..pd_read_csv("/tmp/rxlibs_doc_example.csv"))`
		`=> 1`

`pd_parse_csv(`
`  text = .string`
`)`
`  = .binary`
	Parse CSV text.	`rxpandas..pd_parse_csv("a,b" || "0A"x || "1,x" || "0A"x || "2,y")`
		`=>      a  b`
		`0    1  x`
		`1    2  y`

`pd_to_csv(`
`  df = .binary,`
`  path = .string`
`)`
`  = .int`
	Write a CSV file; returns 0.	`rxpandas..pd_to_csv(df, "/tmp/rxlibs_doc_example.csv")`
		`=> 0`

`pd_add_col(`
`  df = .binary,`
`  name = .string,`
`  values = .binary`
`)`
`  = .int`
	Add or replace a float64 column from an array (in place); returns 0.	`t = rxpandas..pd_new()`
		`ignore = rxpandas..pd_add_col(t, "x", w)`
		`say rxpandas..pd_nrows(t)`
		`=> 3`

`pd_add_str_col(`
`  df = .binary,`
`  name = .string,`
`  csv = .string`
`)`
`  = .int`
	Add a string column from comma separated text (in place); returns 0.	`t = rxpandas..pd_new()`
		`ignore = rxpandas..pd_add_str_col(t, "c", "u,v,u")`
		`say rxpandas..pd_gets(t, "c", 1)`
		`=> v`

`pd_to_string(`
`  df = .binary,`
`  maxrows = .int`
`)`
`  = .string`
	Text rendering like DataFrame.to_string (at most maxrows rows).	`df`
		`=>        city  sales`
		`0    Zurich   10.5`
		`1     Basel   20.0`
		`2    Zurich  30.25`
		`3      Bern    5.0`

`pd_nrows(`
`  df = .binary`
`)`
`  = .int`
	Number of rows.	`rxpandas..pd_nrows(df)`
		`=> 4`

`pd_ncols(`
`  df = .binary`
`)`
`  = .int`
	Number of columns.	`rxpandas..pd_ncols(df)`
		`=> 2`

`pd_colname(`
`  df = .binary,`
`  i = .int`
`)`
`  = .string`
	Name of column i (0-based).	`rxpandas..pd_colname(df, 1)`
		`=> sales`

`pd_isstr(`
`  df = .binary,`
`  name = .string`
`)`
`  = .int`
	1 when the column holds strings.	`rxpandas..pd_isstr(df, "city")`
		`=> 1`

Call	Description	Example and result

`pd_col(`
`  df = .binary,`
`  name = .string`
`)`
`  = .binary`
	Float64 column as an array.	`rxpandas..pd_col(df, "sales")`
		`=> [10.5, 20.0, 30.25, 5.0]`

`pd_get(`
`  df = .binary,`
`  name = .string,`
`  row = .int`
`)`
`  = .float`
	Numeric cell as float.	`rxpandas..pd_get(df, "sales", 2)`
		`=> 30.25`

`pd_gets(`
`  df = .binary,`
`  name = .string,`
`  row = .int`
`)`
`  = .string`
	Cell as string.	`rxpandas..pd_gets(df, "city", 3)`
		`=> Bern`

Figure 33. Creating and inspecting tables

Selecting and ordering

Call	Description	Example and result

`pd_filter(`
`  df = .binary,`
`  col = .string,`
`  op = .string,`
`  value = .string`
`)`
`  = .binary`
	Rows where column op value (op: > < >= <= == !=; strings compare as text).	`rxpandas..pd_filter(df, "sales", ">", "9")`
		`=>        city  sales`
		`0    Zurich   10.5`
		`1     Basel   20.0`
		`2    Zurich  30.25`

`pd_sort(`
`  df = .binary,`
`  col = .string,`
`  ascending = .int`
`)`
`  = .binary`
	Sort by one column.	`rxpandas..pd_sort(df, "sales", 0)`
		`=>        city  sales`
		`2    Zurich  30.25`
		`1     Basel   20.0`
		`0    Zurich   10.5`
		`3      Bern    5.0`

`pd_head(`
`  df = .binary,`
`  n = .int`
`)`
`  = .binary`
	First n rows.	`rxpandas..pd_head(df, 2)`
		`=>        city  sales`
		`0    Zurich   10.5`
		`1     Basel   20.0`

`pd_select(`
`  df = .binary,`
`  names = .string`
`)`
`  = .binary`
	Keep the named columns (comma separated).	`rxpandas..pd_select(df, "sales,city")`
		`=>      sales    city`
		`0     10.5  Zurich`
		`1     20.0   Basel`
		`2    30.25  Zurich`
		`3      5.0    Bern`

`pd_sort_multi(`
`  df = .binary,`
`  cols = .string,`
`  ascending = .string`
`)`
`  = .binary`
	Sort by several columns with per-column ascending flags ("1,0").	`rxpandas..pd_sort_multi(df, "city,sales", "1,0")`
		`=>        city  sales`
		`1     Basel   20.0`
		`3      Bern    5.0`
		`2    Zurich  30.25`
		`0    Zurich   10.5`

Figure 34. Selecting and ordering

Grouping, merging and reshaping

Call	Description	Example and result

`pd_groupby(`
`  df = .binary,`
`  key = .string,`
`  val = .string,`
`  agg = .string`
`)`
`  = .binary`
	Group by one key column and aggregate one value column: sum mean min max count median std.	`rxpandas..pd_groupby(df, "city", "sales", "sum")`
		`=>        city  sales`
		`0     Basel   20.0`
		`1      Bern    5.0`
		`2    Zurich  40.75`

Call	Description	Example and result

`pd_groupby_multi(`
`  df = .binary,`
`  keys = .string,`
`  val = .string,`
`  agg = .string`
`)`
`  = .binary`
	Group by several key columns (comma separated).	`rxpandas..pd_groupby_multi(df, "city", "sales", "mean")`
		`=>  city  sales`
		`0  Basel  20.0`
		`1  Bern  5.0`
		`2  Zurich  20.375`

`pd_merge(`
`  a = .binary,`
`  b = .binary,`
`  key = .string,`
`  how = .string`
`)`
`  = .binary`
	Merge on a key column; how = inner, left, right or outer.	`rxpandas..pd_merge(df, df2, "city", "left")`
		`=>  city  sales  pop`
		`0  Zurich  10.5  420.0`
		`1  Basel  20.0  NaN`
		`2  Zurich  30.25  420.0`
		`3  Bern  5.0  135.0`

`pd_describe(`
`  df = .binary`
`)`
`  = .binary`
	count, mean, std, min, quartiles, max of the numeric columns.	`rxpandas..pd_describe(df)`
		`=>  stat  sales`
		`0  count  4.0`
		`1  mean  16.4375`
		`2  std  11.098751806697305`
		`3  min  5.0`
		`4  25%  9.125`
		`5  50%  15.25`
		`6  75%  22.5625`
		`7  max  30.25`

`pd_value_counts(`
`  df = .binary,`
`  col = .string`
`)`
`  = .binary`
	Counts of the distinct values of a column, most frequent first.	`rxpandas..pd_value_counts(df, "city")`
		`=>  city  count`
		`0  Zurich  2`
		`1  Basel  1`
		`2  Bern  1`

`pd_pivot(`
`  df = .binary,`
`  index = .string,`
`  columns = .string,`
`  values = .string,`
`  agg = .string`
`)`
`  = .binary`
	Pivot table: index, columns, values and an aggregation.	`rxpandas..pd_pivot(df, "city", "city", "sales", "sum")`
		`=>  city  Basel  Bern  Zurich`
		`0  Basel  20.0  NaN  NaN`
		`1  Bern  NaN  5.0  NaN`
		`2  Zurich  NaN  NaN  40.75`

Figure 35. Grouping, merging and reshaping

Index

Call	Description	Example and result

`pd_set_index(`
`  df = .binary,`
`  col = .string`
`)`
`  = .binary`
	Use a column as labelled index.	`rxpandas..pd_set_index(df, "city")`
		`=>  sales`
		`Zurich  10.5`
		`Basel  20.0`
		`Zurich  30.25`
		`Bern  5.0`

`pd_reset_index(`
`  df = .binary,`
`  drop = .int`
`)`
`  = .binary`
	Back to the range index (drop = 1 discards the old labels).	`rxpandas..pd_reset_index(rxpandas..pd_set_index(df, "city"), 1)`
		`=>  sales`
		`0  10.5`
		`1  20.0`
		`2  30.25`
		`3  5.0`

`pd_index_label(`
`  df = .binary,`
`  row = .int`
`)`
`  = .string`
	Index label of row i as text.	`rxpandas..pd_index_label(rxpandas..pd_sort(df, "sales", 1), 0)`
		`=> 3`

`pd_loc(`
`  df = .binary,`
`  label = .string`
`)`
`  = .int`
	Row position of an index label.	`rxpandas..pd_loc(rxpandas..pd_set_index(df, "city"), "Bern")`
		`=> 3`

Figure 36. Index

Date and time

Call	Description	Example and result

`pd_to_datetime(`
`  df = .binary,`
`  col = .string,`
`  newcol = .string`
`)`
`  = .int`
	Parse an ISO date/time string column into epoch seconds in a new column (in place); returns 0.	`t = rxpandas..pd_parse_csv("d" || "0A"x || "2024-03-15 10:30:00")`
		`ignore = rxpandas..pd_to_datetime(t, "d", "e")`
		`say rxpandas..pd_get(t, "e", 0)`
		`=> 1710498600`

`pd_dt_part(`
`  df = .binary,`
`  epoch_col = .string,`
`  part = .string,`
`  newcol = .string`
`)`
`  = .int`
	Date part (year month day hour minute second dow doy) of an epoch column into a new column (in place).	`t = rxpandas..pd_parse_csv("d" || "0A"x || "2024-03-15 10:30:00")`
		`ignore = rxpandas..pd_to_datetime(t, "d", "e")`
		`ignore = rxpandas..pd_dt_part(t, "e", "dow", "w")`
		`say rxpandas..pd_get(t, "w", 0)`
		`=> 4`

`pd_parse_datetime(`
`  text = .string`
`)`
`  = .float`
	ISO date/time text to epoch seconds.	`rxpandas..pd_parse_datetime("2024-03-15 10:30:00")`
		`=> 1710498600`

`pd_datetime_part(`
`  epoch = .float,`
`  part = .string`
`)`
`  = .int`
	Part of an epoch value.	`rxpandas..pd_datetime_part(1710498600.0, "month")`
		`=> 3`

Figure 37. Date and time

Chapter 7. Callbacks and the Solver Reference

The Level B callback convention

A native plugin cannot call back into a Level B procedure, and Level B has no first-class procedure values, but it has interfaces and classes. The solvers are therefore Level B procedures in the module rxspalg, written on top of the rxnumpy and rxscipy plugins. The objective is an object: the caller writes a class that implements one of the library interfaces, creates an instance and passes it; the solver calls its method.

Interface	Method	Used by
.sp_f1	f(x = .float) = .float	sp_brentq, sp_newton, sp_quad
.sp_f1d	f(x), df(x)	sp_newton_d
.sp_fn	f(x = .binary) = .float; x is an rxnumpy float64 vector	sp_minimize_nm, sp_minimize_bfgs
.sp_ode	rhs(t = .float, y = .binary) = .binary	sp_solve_ivp
.sp_model	f(x = .float, p = .binary) = .float	sp_curve_fit

Figure 38. Callback interfaces of rxspalg

```
import rxnumpy
import rxspalg
cube: class implements .sp_f1
  *: factory
    return
  f: method = .float
  arg x = .float
  return x * x * x - 2.0 * x - 5.0
main: procedure = .int
  say rxspalg..sp_brentq(.cube(), 2.0, 3.0, 0.000000000002, 0.000000000000000888, 100)
  return 0
```

Figure 39. A callback class (from the repository tests)

The class needs a factory (the line *: factory followed by return). The vector given to a callback is shared by reference and belongs to the solver: a callback must not keep or modify it. The rxspalg library is compiled with buildlib.sh to build/lib/rxspalg.rxbin; programs compile with -i build/lib and run with -l build/lib/rxspalg.rxbin (rxrun does both).

Solver reference

Call	Description

`sp_brentq(`
`  f = .sp_f1,`
`  xa = .float,`
`  xb = .float,`
`  xtol = .float,`
`  rtol = .float,`
`  maxiter = .int`
`)`
`  = .float`
	Root of f in [xa, xb] with sign change (Brent, as scipy brentq.c).

`sp_newton(`
`  f = .sp_f1,`
`  x0 = .float,`
`  tol = .float,`
`  maxiter = .int`
`)`
`  = .float`
	Secant iteration from x0.

`sp_newton_d(`
`  f = .sp_f1d,`
`  x0 = .float,`
`  tol = .float,`
`  maxiter = .int`
`)`
`  = .float`
	Newton iteration with derivative method df.

Call	Description

`sp_quad(`
`  f = .sp_f1,`
`  a = .float,`
`  b = .float,`
`  epsabs = .float,`
`  epsrel = .float,`
`  limit = .int,`
`  expose abserr = .float`
`)`
`  = .float`
	Adaptive Gauss-Kronrod (21 points) integral on [a, b]; abserr is the error estimate.

`sp_minimize_nm(`
`  f = .sp_fn,`
`  x0 = .binary,`
`  xatol = .float,`
`  fatol = .float,`
`  maxiter = .int,`
`  expose fval = .float,`
`  expose nfev = .int`
`)`
`  = .binary`
	Nelder-Mead simplex (scipy variant); returns the minimiser.

`sp_minimize_bfgs(`
`  f = .sp_fn,`
`  x0 = .binary,`
`  gtol = .float,`
`  maxiter = .int,`
`  expose fval = .float,`
`  expose nit = .int`
`)`
`  = .binary`
	BFGS with a central-difference gradient and Armijo backtracking.

`sp_solve_ivp(`
`  f = .sp_ode,`
`  t0 = .float,`
`  t1 = .float,`
`  y0 = .binary,`
`  t_eval = .binary,`
`  rtol = .float,`
`  atol = .float,`
`  expose tout = .binary,`
`  expose nfev = .int`
`)`
`  = .binary`
	RK45 (Dormand-Prince) with dense output; returns y as an (n, m) array at the times tout.

`sp_curve_fit(`
`  f = .sp_model,`
`  x = .binary,`
`  y = .binary,`
`  p0 = .binary,`
`  maxiter = .int,`
`  expose pcov = .binary,`
`  expose cost = .float`
`)`
`  = .binary`
	Levenberg-Marquardt least squares; returns the parameters, pcov = inv(J'J) * cost / (m - n).

Figure 40. The solvers of rxspalg

- **sp_brentq** is brentq.c of scipy; **sp_newton** is the secant method (or Newton with the derivative in sp_newton_d).
- **sp_quad** is an adaptive Gauss-Kronrod 21-point rule with the QUADPACK qk21 error estimate and bisection of the interval with the largest error. There are no infinite limits and no extrapolation (the epsilon algorithm of qags).
- **sp_minimize_nm** is scipy's Nelder-Mead (initial simplex 5 percent, stable sort). maxiter limits both the iterations and the function evaluations; with maxiter 0 the scipy default of 200 n is used. **sp_minimize_bfgs** uses a central difference gradient and an Armijo backtracking line search: the minimum is the same as scipy's, the iterates are not.
- **sp_solve_ivp** is RK45 with scipy's initial step, step control (safety 0.9, factors 0.2 to 10) and the fourth order dense output, so that the default step sequence is identical to scipy's. t_eval may be an empty array (np_zeros("0")) to get the solver's own steps. The result is an array of shape (n, m): one row per component, one column per time.
- **sp_curve_fit** is Levenberg-Marquardt with a central difference Jacobian; pcov = inv(J'J) * cost / (m - n), like scipy.

Chapter 8. Worked Examples

Linear regression

The data are a straight line plus noise from numpy's default_rng(1), which the plugin reproduces. The slope and intercept are found three ways: through the normal equations with np_solve, with sp_polyfit and with sp_linregress, which also gives r, the p-value and the standard errors. The program ex_linreg.py in the repository runs the same computation in CPython with numpy and scipy; its output is identical to the lines below.

```
options levelb floats_binary
import rxfnsb
import rxnumpy
import rxscipy

r6: procedure = .string
  arg x = .binary
  return rxnumpy..np_tolist(rxnumpy..np_divs(rxnumpy..np_round(rxnumpy..np_muls(x,
      1000000.0)), 1000000.0))

main: procedure = .int
  /* data: y = 2.5 + 0.8 x + noise, noise from numpy's default_rng(1) */
  rng = rxnumpy..np_default_rng(1)
  x = rxnumpy..np_arangef(0.0, 10.0, 1.0)
  noise = rxnumpy..np_rng_normal(rng, 0.0, 0.3, "10")
  y = rxnumpy..np_add(rxnumpy..np_adds(rxnumpy..np_muls(x, 0.8), 2.5), noise)
  /* 1. normal equations (X'X) beta = X'y with the design matrix [1, x] */
  design = rxnumpy..np_transpose(rxnumpy..np_vstack(rxnumpy..np_ones("10"), x))
  xt = rxnumpy..np_transpose(design)
  beta = rxnumpy..np_solve(rxnumpy..np_dot(xt, design), rxnumpy..np_dot(xt, y))
  say "normal equations [intercept, slope]:" r6(beta)
  /* 2. polyfit, highest power first */
  say "polyfit          [slope, intercept]:" r6(rxscipy..sp_polyfit(x, y, 1))
  /* 3. linregress: slope, intercept, r, p, stderr, intercept_stderr */
  say "linregress:" r6(rxscipy..sp_linregress(x, y))
  /* residual standard deviation and R squared */
  fit = rxnumpy..np_adds(rxnumpy..np_muls(x, rxnumpy..np_get(beta, 1)),
      rxnumpy..np_get(beta, 0))
  res = rxnumpy..np_sub(y, fit)
  ss_res = rxnumpy..np_sum(rxnumpy..np_square(res))
  ss_tot = rxnumpy..np_sum(rxnumpy..np_square(rxnumpy..np_subs(y, rxnumpy..np_mean(y))))
  say "R squared:" r6(rxnumpy..np_scalar(1.0 - ss_res / ss_tot))
  return 0

$ rxrun ex_linreg.crexx
normal equations [intercept, slope]: [2.572131, 0.798965]
polyfit          [slope, intercept]: [0.798965, 2.572131]
linregress: [0.798965, 2.572131, 0.996634, 0.0, 0.023236, 0.124048]
R squared: 0.993279
```

Figure 41. Linear regression

A differential equation

A chain of two first order reactions, y0' = -0.5 y0 and y1' = y0 - y1, is integrated from t = 0 to 4 with sp_solve_ivp. The right-hand side is the method rhs of a class that implements .sp_ode. The solution is compared with the analytic one.

```
options levelb floats_binary
import rxfnsb
import rxnumpy
import rxscipy
import rxspalg

/* y0' = -0.5 y0,  y1' = y0 - y1 : a chain of two first order reactions */
chain: class implements .sp_ode
  *: factory
    return
  rhs: method = .binary
  arg t = .float, y = .binary
  y0 = rxnumpy..np_get(y, 0)
  y1 = rxnumpy..np_get(y, 1)
  return rxnumpy..np_array("[" || (0.0 - 0.5 * y0) || "," || (y0 - y1) || "]")

r6: procedure = .string
  arg x = .binary
  return rxnumpy..np_tolist(rxnumpy..np_divs(rxnumpy..np_round(rxnumpy..np_muls(x,
      1000000.0)), 1000000.0))

main: procedure = .int
  y0 = rxnumpy..np_array("[1.0, 0.0]")
  want = rxnumpy..np_array("[0.0, 1.0, 2.0, 4.0]")
  tout = rxnumpy..np_zeros("1")
  nfev = 0
  ys = rxspalg..sp_solve_ivp(.chain(), 0.0, 4.0, y0, want, 0.0000000001, 0.000000000001,
      tout, nfev)
  say "t       " r6(tout)
  say "y0      " r6(rxnumpy..np_slice(ys, 0, 0, 1, 1))
  say "y1      " r6(rxnumpy..np_slice(ys, 0, 1, 2, 1))
  /* exact: y0 = exp(-t/2), y1 = 2 (exp(-t/2) - exp(-t)) */
  exact0 = rxnumpy..np_exp(rxnumpy..np_muls(want, -0.5))
  exact1 = rxnumpy..np_muls(rxnumpy..np_sub(exact0,
      rxnumpy..np_exp(rxnumpy..np_neg(want))), 2.0)
  say "exact y1" r6(exact1)
  /* default time points: the step sequence chosen by the integrator (identical to
      scipy's RK45) */
  ys = rxspalg..sp_solve_ivp(.chain(), 0.0, 4.0, y0, rxnumpy..np_zeros("0"), 0.001,
      0.000001, tout, nfev)
  say "steps:" rxnumpy..np_size(tout) "function evaluations:" nfev
  return 0

$ rxrun ex_ode.crexx
t        [0.0, 1.0, 2.0, 4.0]
y0       [[1.0, 0.606531, 0.367879, 0.135335]]
y1       [[0.0, 0.477302, 0.465088, 0.234039]]
exact y1 [0.0, 0.477302, 0.465088, 0.234039]
steps: 9 function evaluations: 50
```

Figure 42. Solving a system of differential equations

Optimisation

A root of cos(x) = x by Brent's method, the minimum of the Rosenbrock function by Nelder-Mead and by BFGS, and a least squares fit of an exponential decay with standard errors from the covariance matrix.

```
options levelb floats_binary
import rxfnsb
import rxnumpy
import rxscipy
import rxspalg

ross: class implements .sp_fn
  *: factory
    return
  f: method = .float
  arg x = .binary
  a = rxnumpy..np_get(x, 0)
  b = rxnumpy..np_get(x, 1)
  return (1.0 - a) * (1.0 - a) + 100.0 * (b - a * a) * (b - a * a)

/* model p0 * exp(-p1 x) + p2 */
expm: class implements .sp_model
  *: factory
    return
  f: method = .float
  arg x = .float, p = .binary
  return rxnumpy..np_get(p, 0) * rxscipy..sp_exp(0.0 - rxnumpy..np_get(p, 1) * x) +
      rxnumpy..np_get(p, 2)

cosm: class implements .sp_f1
  *: factory
    return
  f: method = .float
  arg x = .float
  return rxscipy..sp_cos(x) - x

r6: procedure = .string
  arg x = .binary
  return rxnumpy..np_tolist(rxnumpy..np_divs(rxnumpy..np_round(rxnumpy..np_muls(x,
      1000000.0)), 1000000.0))

main: procedure = .int
  /* root finding: cos(x) = x */
  say "brentq:" rxspalg..sp_brentq(.cosm(), 0.0, 1.0, 0.000000000001, 0.000000000000001,
      100)
  /* minimisation of the Rosenbrock function from (-1.2, 1) */
  x0 = rxnumpy..np_array("[-1.2, 1.0]")
  fval = 0.0
  nfev = 0
  nit = 0
  xm = rxspalg..sp_minimize_nm(.ross(), x0, 0.00000001, 0.00000001, 0, fval, nfev)
  say "Nelder-Mead:" r6(xm) "f =" r6(rxnumpy..np_scalar(fval))
  xb = rxspalg..sp_minimize_bfgs(.ross(), x0, 0.00000001, 0, fval, nit)
  say "BFGS:       " r6(xb) "f =" r6(rxnumpy..np_scalar(fval))
  /* curve fit of an exponential decay with offset */
  xd = rxnumpy..np_array("[0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0]")
  yd = rxnumpy..np_array("[5.2, 3.9, 3.0, 2.5, 2.1, 1.9, 1.6, 1.55, 1.4]")
  pcov = rxnumpy..np_zeros("1")
  cost = 0.0
  p = rxspalg..sp_curve_fit(.expm(), xd, yd, rxnumpy..np_array("[4.0, 1.0, 1.0]"), 0,
      pcov, cost)
  say "curve_fit p:" r6(p)
  say "std errors: " r6(rxnumpy..np_sqrt(rxnumpy..np_array("[" || rxnumpy..np_get2(pcov,
      0, 0) || "," || rxnumpy..np_get2(pcov, 1, 1) || "," || rxnumpy..np_get2(pcov, 2,
      2) || "]")))
```

```
  return 0

$ rxrun ex_optim.crexx
brentq: 0.739085133215156
Nelder-Mead: [1.0, 1.0] f = 0.0
BFGS:        [1.0, 1.0] f = 0.0
curve_fit p: [3.906233, 0.787484, 1.277243]
std errors:  [0.057542, 0.031955, 0.050226]
```

Figure 43. Root finding, minimisation and curve fitting

Dataframe groupby

A small sales table is read from CSV text, a column is added, and the table is grouped, pivoted, counted and merged.

```
options levelb floats_binary
import rxfnsb
import rxnumpy
import rxpandas

main: procedure = .int
  nl = "0A"x
  csv = "region,product,units,price" || nl || "north,apple,10,1.5" || nl || "south,
      apple,4,1.6" || nl || "north,pear,7,2.0" || nl,
     || "east,pear,12,1.9" || nl || "south,plum,3,3.1" || nl || "north,apple,5,1.4" ||
         nl || "east,apple,8,1.5"
  df = rxpandas..pd_parse_csv(csv)
  say rxpandas..pd_to_string(df, 20)
  /* revenue column = units * price */
  call rxpandas..pd_add_col(df, "revenue", rxnumpy..np_mul(rxpandas..pd_col(df,
      "units"), rxpandas..pd_col(df, "price")))
  say "revenue by region:"
  say rxpandas..pd_to_string(rxpandas..pd_groupby(df, "region", "revenue", "sum"), 20)
  say "mean units by region and product:"
  say rxpandas..pd_to_string(rxpandas..pd_groupby_multi(df, "region,product", "units",
      "mean"), 20)
  say "pivot (revenue):"
  say rxpandas..pd_to_string(rxpandas..pd_pivot(df, "region", "product", "revenue",
      "sum"), 20)
  say "value counts of product:"
  say rxpandas..pd_to_string(rxpandas..pd_value_counts(df, "product"), 20)
  names = rxpandas..pd_parse_csv("region,manager" || nl || "north,Ines" || nl || "south,
      Paul" || nl || "west,Kim")
  say "merge (left):"
  say rxpandas..pd_to_string(rxpandas..pd_select(rxpandas..pd_merge(rxpandas..pd_groupby
      (df, "region", "revenue", "sum"), names, "region", "left"), "region,revenue,
      manager"), 20)
  return 0

$ rxrun ex_groupby.crexx
     region  product  units  price
0     north    apple     10    1.5
1     south    apple      4    1.6
2     north     pear      7    2.0
3      east     pear     12    1.9
4     south     plum      3    3.1
5     north    apple      5    1.4
6      east    apple      8    1.5

revenue by region:
     region             revenue
0      east                34.8
1     north                36.0
2     south  15.700000000000001

mean units by region and product:
     region  product  units
0      east    apple    8.0
1      east     pear   12.0
2     north    apple    7.5
3     north     pear    7.0
4     south    apple    4.0
5     south     plum    3.0

pivot (revenue):
```

```
     region  apple                pear  plum
0      east   12.0  22.799999999999997   NaN
1     north   22.0                14.0   NaN
2     south    6.4                 NaN   9.3

value counts of product:
     product  count
0      apple      4
1       pear      2
2       plum      1

merge (left):
     region             revenue  manager
0      east                34.8
1     north                36.0     Ines
2     south  15.700000000000001     Paul
```

Figure 44. Groupby, pivot, value_counts and merge

Statistical tests and distributions

Hypothesis tests and distribution functions. The Python program ex_stats.py of the repository prints the same lines with scipy.stats.

```
options levelb floats_binary
import rxfnsb
import rxnumpy
import rxscipy

r6: procedure = .string
  arg x = .float
  return rxnumpy..np_tolist(rxnumpy..np_scalar(rxnumpy..np_get(rxnumpy..np_divs(rxnumpy.
      .np_round(rxnumpy..np_muls(rxnumpy..np_scalar(x), 1000000.0)), 1000000.0), 0)))

main: procedure = .int
  a = rxnumpy..np_array("[5.1, 4.9, 6.2, 5.8, 6.0, 5.5, 5.3, 6.1]")
  b = rxnumpy..np_array("[6.3, 6.8, 5.9, 7.1, 6.6, 6.9, 7.4, 6.2]")
  t = 0.0
  p = 0.0
  df = 0.0
  call rxscipy..sp_ttest_1samp(a, 5.0, t, p)
  say "one sample t test vs 5.0:  t =" r6(t) " p =" r6(p)
  call rxscipy..sp_ttest_ind(a, b, t, p)
  say "two sample t test:         t =" r6(t) " p =" r6(p)
  call rxscipy..sp_ttest_welch(a, b, t, p, df)
  say "Welch t test:              t =" r6(t) " p =" r6(p) " df =" r6(df)
  /* chi-square test of independence for a 2 x 3 table */
  tab = rxnumpy..np_array("[[20, 15, 30], [10, 25, 20]]")
  chi = 0.0
  dof = 0
  expected = rxnumpy..np_zeros("1")
  call rxscipy..sp_chi2_contingency(tab, 1, chi, p, dof, expected)
  say "chi2 =" r6(chi) " p =" r6(p) " dof =" dof
  /* critical values and tail probabilities */
  say "t quantile (df 7, 0.975):" r6(rxscipy..sp_ppf("t", 0.975, 7.0, 0.0, 0.0, 1.0))
  say "P(chi2(4) > 9.49):       " r6(rxscipy..sp_sf("chi2", 9.49, 4.0, 0.0, 0.0, 1.0))
  say "binomial(20, 0.3) P(X <= 8):" r6(rxscipy..sp_cdf("binom", 8.0, 20.0, 0.3, 0.0,
      1.0))
  return 0

$ rxrun ex_stats.crexx
one sample t test vs 5.0:  t = 3.569626  p = 0.009101
two sample t test:         t = -4.217571  p = 0.000861
Welch t test:              t = -4.217571  p = 0.000862  df = 13.989856
chi2 = 7.048951  p = 0.029467  dof = 2
t quantile (df 7, 0.975): 2.364624
P(chi2(4) > 9.49):        0.049953
binomial(20, 0.3) P(X <= 8): 0.886669
```

Figure 45. t tests, chi-square test and distribution functions

Chapter 9. Limits

What is not implemented

- No complex dtype; only float64 and int64 arrays (masks are int64 0/1). No int axis tuples in reductions.
- Generator: random, integers, normal, exponential, choice (1-d, no p), shuffle, permutation. No other distributions on the default_rng stream; the other distributions are available on the legacy stream through sp_rvs.
- sp_quad has no infinite limits and no singularity extrapolation (qagi, qags). Only unconstrained optimisers: no L-BFGS-B or bounds. Only the explicit RK45 ODE solver; no stiff solvers. sp_interp is linear and sp_cubic_spline is cubic; no other interpolation kinds.
- BFGS has a central difference gradient and a backtracking line search: the same minimum as scipy but different iterates and counts. Nelder-Mead, RK45, brentq and curve_fit follow scipy iteration for iteration.
- rxpandas: float64 and string columns only; no categoricals, no string methods, no time zones, ISO date and time only. Aggregations are sum, mean, min, max, count, median and std.
- Comparisons treat inf - inf as unequal.
- Thread safety: the error buffers and the global random generators are per library and not thread safe.
- A handle must not outlive its plugin library; arrays are limited by memory (malloc) and by the 8 dimensions of rxnp.
- sp_rvs: binom with n*min(p, 1-p) above 30 and poisson with mu of 10 or more are correct in distribution but not identical to numpy (Chapter 5).

Numerical agreement

Special functions and distribution functions agree with scipy to about 1e-13 relative error on the test grid of Appendix C. Sums, random streams and sorting agree exactly. Matrix products and the linear algebra kernels are straightforward loops and agree to rounding error, not bit for bit with LAPACK. The test programs compare after rounding to 6 to 9 decimals.

Chapter 10. Quirks of Level B

These are the pitfalls found while writing the libraries and the examples of this manual.

- Write helper procedures and a procedure main: procedure = .int. Code after helper procedures outside main is not run.
- Variables first assigned inside a do block are local to that block. Assign the variable before the loop when it is used after the loop (this matters for loop results and for out parameters).
- Floating point arguments and literals need options levelb floats_binary.
- // starts a comment in Level B; it is not the remainder operator.
- fn is not usable as a variable name.
- A class that implements a library interface needs a factory: a line *: factory followed by return. A method is declared with name: method = .type and arg lines.
- The compiler driver refuses to build when a log file is written inside the compile directory; run.sh uses a scratch directory.
- The driver writes .rxbin and .rxas files next to the source, and the plugin directory is needed twice: with -i at compile time and with -l at run time.
- Out parameters are expose arguments: declare and initialise the variable first, with the type of the argument.
- A signal handler is part of the do block: on signal name as problem follows the statements that may fail, inside the same do ... end.
- Handles are references: h = g does not copy the array. Use np_copy before np_set when the original must not change.

Chapter 11. Performance

The benchmark program bench/run_bench.py times each operation on the plugin and on CPython (numpy, scipy, pandas) for several sizes. The runs are serialised: CPython first, then the CREXX program, one case after the other. Each case is timed in 5 repetitions of N calls; the table gives the median repetition divided by N. N is chosen per case so that the slower side takes about 30 ms per repetition (N = 1 for the slowest cases). The data are created outside the timed region. BLAS is limited to one thread, since the plugins are single-threaded. The raw results are in bench/results.csv.

The machine is a shared two-core Linux virtual machine, so absolute times vary by tens of percent from run to run; the ratios are what matters. The CREXX side includes the cost of the CREXX call (argument conversion and handle creation), the CPython side includes the cost of the Python call and of numpy's dispatch. The CREXX clock (time('e')) has a resolution of a millisecond on the cases where N is 1.

Operation	Size	N	Plugin	CPython	Plugin / CPython
matmul	50x50	107	101.0 us	14.4 us	7.01
matmul	100x100	35	772.3 us	49.4 us	15.62
matmul	200x200	2	6.07 ms	781.5 us	7.76
sum	1000	1578	1.3 us	3.0 us	0.44
sum	100000	297	47.6 us	34.0 us	1.40
sum	1000000	25	1.79 ms	485.0 us	3.69
fft	1024	333	49.1 us	41.7 us	1.18
fft	16384	17	1.66 ms	500.6 us	3.32
fft	262144	1	43.20 ms	13.11 ms	3.30
sort	1000	309	79.3 us	7.6 us	10.39
sort	100000	1	16.80 ms	1.02 ms	16.52
sort	1000000	1	313.40 ms	19.59 ms	16.00
solve	50x50	57	533.2 us	36.3 us	14.67
solve	200x200	1	35.38 ms	566.5 us	62.45
solve	500x500	1	615.13 ms	6.41 ms	95.93
groupby	1000 rows/100 keys	44	124.3 us	462.8 us	0.27
groupby	100000 rows/100 keys	1	17.95 ms	4.41 ms	4.07
groupby	1000000 rows/100 keys	1	307.52 ms	21.40 ms	14.37
quad	cos(1x) on [0,10]	1	133.48 ms	7.0 us	19055
quad	cos(10x) on [0,10]	1	106.64 ms	49.9 us	2135
quad	cos(100x) on [0,10]	1	108.58 ms	832.3 us	130
minimize NM	rosenbrock n=2	2	10.14 ms	10.09 ms	1.00
minimize BFGS	rosenbrock n=2	1	3.81 ms	13.95 ms	0.27
minimize NM	rosenbrock n=5	1	117.27 ms	23.35 ms	5.02
minimize BFGS	rosenbrock n=5	1	11.39 ms	31.28 ms	0.36
minimize NM	rosenbrock n=10	1	2.44 s	149.89 ms	16.27
minimize BFGS	rosenbrock n=10	1	55.47 ms	52.55 ms	1.05

Figure 46. Plugin calls against CPython numpy, scipy and pandas

Reading the table

- **Array kernels** (sum, fft, sort, matmul, solve) are simple C loops, single-threaded, without SIMD. numpy uses vectorised kernels and, for matmul and solve, an optimised BLAS/LAPACK: the plugin is 7 to 16 times slower for matmul and 15 to 96 times slower for solve, 1.2 to 3.3 times for fft and 10.4 to 16.5 times for sort. For small arrays the call overhead dominates and the gap is smaller; sum of 1000 elements is faster in the plugin than in numpy (0.44 of the time).
- **groupby** on a small table is faster in the plugin (the pandas overhead for a groupby is about half a millisecond); on a million rows pandas is 14 times faster.

- **quad** is a Level B algorithm calling back a Level B method for each of the 21 nodes of every Gauss-Kronrod rule; it runs at the speed of the CREXX virtual machine and is far slower than the compiled QUADPACK of scipy. Use it for the occasional integral, not in a loop.
- **minimize** (Nelder-Mead and BFGS) is Level B code over native vector operations. Both Nelder-Mead runs converge to the same tolerance; the plugin needs 1.0 to 16.3 times as long, the more the larger the dimension, because every function evaluation is a Level B callback with several plugin calls. The two BFGS implementations do different work (central against forward differences, Armijo against Wolfe line search), so their times (plugin / CPython 0.27 to 1.05) are not a like for like comparison.
- Against the Python/CREXX interpreter the plugins are far faster (the README lists 220 to 43000 times for matmul 100x100, sum of 10^6 and fft 1024); the table above compares with real CPython.

Appendix A. Test Programs

The 9 test programs are run by scripts/runtests.sh. Each compares the CREXX output with the output of CPython with numpy 2.5.3, scipy 1.18.1 and pandas 3.0.5, after rounding floating point numbers to 6 to 9 decimals. The number of output lines (cases) is given for each.

Program	Lines compared	Covers
rxnumpy/tests/t_basic.crexx	34	rxnumpy: elementwise operations, broadcasting, reductions, slicing, sort, dot, error signals
rxnumpy/tests/t_fft_random.crexx	8	rxnumpy: FFT, legacy MT19937 random streams
rxnumpy/tests/t_linalg.crexx	11	rxnumpy: solve, inv, det, cholesky, qr, eigh, svd, eigvals, singular matrix signal
rxnumpy/tests/t_pcg_idx.crexx	36	rxnumpy: default_rng streams bit for bit, boolean and integer indexing, where, unique, stack, n-d matmul
rxpandas/tests/t_pandas.crexx	64	rxpandas: CSV, filter, sort, head, select, groupby, merge, describe, indexing, errors
rxpandas/tests/t_pandas2.crexx	74	rxpandas: multi-key groupby and sort, merge variants, value_counts, pivot, index, datetime
rxscipy/tests/t_opt.crexx	12	rxspalg: brentq, newton, quad, Nelder-Mead, BFGS, solve_ivp (incl. the default RK45 step sequence), curve_fit
rxscipy/tests/t_scipy.crexx	15	rxscipy: erf, gamma, normal, t, betainc, trapz, simpson, interp, polyfit, pearsonr, ttest_ind, cubic splines
rxscipy/tests/t_stats.crexx	136	rxscipy: gamma, erf, erfinv, betainc, gammainc; pdf/cdf/sf/ppf/mean/var of 12 distributions; rvs of 16 parameter sets; describe, skew, kurtosis, zscore, percentile, correlate, t tests, spearmanr, linregress, chi2_contingency

Figure 47. Test programs

Total: 9 programs, 390 compared lines. In addition examples_check.py re-runs every listing of this manual and compares its output with the printed one; the two CPython twin programs ex_linreg.py and ex_stats.py must print the same lines as their CREXX versions.

Appendix B. Files of the Repository

```
rxlibs/
  README.md                    overview, API tables, speed
  docs/DESIGN.md               design: handle model, errors, callbacks, roadmap
  docs/Rxlibs-Guide.docx|pdf   this manual
  core/                        C89 libraries (no CREXX, no Python)
    rxnp.[ch] rxnp_la.c rxnp_lapack.[ch] rxnp_fft.c rxnp_rng.c rxnp_pcg.c rxnp_idx.c
        arrays, linalg, FFT, random
    rxsp.[ch] rxsp_dist.c      special functions, distributions, statistics
    rxdf.[ch]                  tables
    rxlibs_handle.h            handle glue shared by the plugins
  rxnumpy/ rxscipy/ rxpandas/  plugin/<name>.c, examples/, tests/ (t_*.crexx,
      *.expected, gen_*.py)
  rxscipy/lib/rxspalg.crexx    Level B callback solvers
  bench/                       run_bench.py, results.csv, accuracy.py, accuracy_driver.c
  scripts/                     build.sh buildlib.sh run.sh runtests.sh gen_api.py
```

Figure 48. Layout of rxlibs

Appendix C. Accuracy Against scipy

The special functions and the distributions were compared with scipy 1.18.1 on a grid of 1143 points: gamma and lgamma (also for negative arguments), erf and erfc (including the far tail), erfinv, beta, betainc, gammainc and gammaincc, and for each of the 12 distributions pdf or pmf, cdf, sf at several points and ppf at the probabilities 1e-6, 0.01, 0.25, 0.5, 0.9, 0.999 and 1 - 1e-9, with and without loc and scale.

Quantity	Value
points compared	1143
points with relative error above 1e-12	0
largest relative error	1.1e-13

Figure 49. Result of the comparison

The comparison is bench/accuracy.py (with bench/accuracy_driver.c) in the repository; the committed test t_stats compares a smaller set of values after rounding to 9 decimals, together with the random streams of 16 parameter sets.